This shared conversation is generated by AI, for reference only.

[KONTEKS PROYEK].txt
TXT 4.64KB

SPEC.md — Spesifikasi Teknis BERT-from-Scratch

> **Status:** Draft v1.0 — menunggu rapat kelompok
> **Peran penyusun:** Ketua Kelompok / Prompt Engineer & LLM Navigator
> **Framework:** PyTorch
> **Metode:** Vibe Coding (kolaborasi aktif dengan LLM, tanpa copy-paste kode utuh)
> **Hardware asumsi:** Google Colab T4 (16GB VRAM) — *catatan: bagian* `[HARDWARE]` *di konteks proyek masih kosong; angka di dokumen ini diasumsikan untuk T4. Jika hardware final berbeda (mis. laptop GPU 4GB / CPU only), angka di §3 dan §5 wajib direvisi lewat rapat sebelum coding dimulai.*

1. Task Model

Task utama yang direkomendasikan: Masked Language Modeling (MLM) murni.

- **Definisi task:** Diberikan kalimat dengan sebagian token diganti `[MASK]`, model memprediksi token asli pada posisi yang di-mask. Ini adalah pre-training objective asli BERT dan paling selaras dengan arsitektur encoder dua arah yang menjadi inti proyek.
- **Contoh input-output konkret:**
  - Input: `"the cat sat on the [MASK] and purred"`
  - Target: token asli di posisi `[MASK]` → `"mat"` (atau token apa pun yang valid secara konteks)
  - Output model: distribusi probabilitas atas `vocab_size` untuk setiap posisi yang di-mask.
- **Metrik evaluasi:**
  - **Masked LM Accuracy** (akurasi top-1 pada posisi yang di-mask) — metrik utama.
  - **Masked LM Loss** (cross-entropy, `ignore_index=-100`) — metrik pendamping untuk memantau konvergensi.

- **Perplexity** pada validation set (turunan dari loss) — untuk perbandingan antar-run.

- **Dataset yang cocok:** WikiText-2 (lihat §2). Alternatif: WikiText-103 (terlalu berat untuk T4 dalam waktu wajar), BookCorpus (lisensi & ukuran tidak praktis).
- **2 task alternatif yang dipertimbangkan:**

  1. **Sentiment Analysis (fine-tuning klasifikasi):** butuh label, menambah kompleksitas data pipeline & head klasifikasi; cocok sebagai *extension* setelah MLM berjalan, bukan task utama.
  2. **Next-Word Prediction / Causal LM:** tidak cocok untuk BERT dua arah (BERT bukan model autoregresif); akan memaksa arsitektur decoder yang menyimpang dari tujuan proyek "BERT dari nol".

> **Keputusan:** MLM murni. Sentiment analysis dicatat sebagai *stretch goal* opsional jika waktu tersisa, tetapi tidak masuk SPEC v1.

2. Dataset & Tokenizer

- **Nama dataset final: WikiText-2** (varian `wikitext-2-raw-v1`).
  - **Split:** train / validation / test (test dipakai sebagai held-out evaluasi akhir; validation untuk monitoring selama training).
  - **Sumber:** HuggingFace Datasets (`wikitext`), lisensi CC BY-SA.
  - **Ukuran perkiraan:** ~2M token total; ~36k baris train, ~3.7k baris validation, ~4.3k baris test.
- **Jenis tokenizer: Word-level tokenizer kustom** (dibangun dari nol oleh anggota Data Pipeline), bukan subword BPE.
  - **Alasan:** lebih sederhana untuk Vibe Coding, mudah di-debug, dan cukup untuk dataset ~2M token. BPE bisa diusulkan sebagai revisi SPEC v2 jika OOV terlalu tinggi.
- `vocab_size` **: 30.000** (target; final ditentukan setelah analisis frekuensi token oleh Data Pipeline — nilai ini masuk kategori perubahan (a) angka/desain).
- `max_seq_length` **: 128** token.
- **Special tokens & ID (ditetapkan di awal, tidak boleh berubah tanpa rapat):**

Token	ID	Keterangan
`[PAD]`	0	padding
`[UNK]`	1	unknown token (OOV)
`[CLS]`	2	klasifikasi / awal sequence
`[SEP]`	3	pemisah sequence
`[MASK]`	4	token masking untuk MLM

> ID 0–4 **dikunci**. Token kosakata lain dimulai dari ID 5.

3. Arsitektur Model (angka saja, bukan kode)

Komponen	Nilai
`hidden_size`	256
`num_layers`	6
`num_heads`	8
`head_dim`	32 (= 256 / 8) ✓ habis dibagi
`intermediate_size`	1024 (4 × hidden)
`dropout`	0.1
`activation`	GELU
`max_seq_length`	128
`vocab_size`	30.000
`type_vocab_size`	2 (segment A/B)

- **MLM Head:** linear projection `hidden_size → hidden_size` + GELU + LayerNorm, lalu linear `hidden_size → vocab_size`, dengan weight tying opsional ke embedding (diputuskan di rapat; default: **tidak** weight-tying agar debugging lebih transparan).
- **Perkiraan jumlah parameter:**
  - Embedding: `30.000 × 256 ≈ 7.7M`
  - Per EncoderLayer: attention ≈ `4 × 256² ≈ 0.26M`, FFN ≈ `2 × 256 × 1024 ≈ 0.52M`, total ≈ **0.8M/layer** × 6 ≈ **4.8M**
  - MLM Head: `256 × 256 + 256 × 30.000 ≈ 7.7M`
  - **Total ≈ 20–21 juta parameter** (tergantung weight tying).

4. Interface Antar Modul

- **Nama variabel input model (wajib, tidak boleh diganti):**
  - `input_ids` — `torch.LongTensor`
  - `attention_mask` — `torch.LongTensor` (1 = token valid, 0 = padding)
  - `labels` — `torch.LongTensor` (token asli di posisi mask; `-100` di posisi non-mask)
  - `token_type_ids` — `torch.LongTensor` (opsional; default nol)
- **Shape tensor di setiap boundary:**

Boundary	Tensor	Shape	dtype
`data.py → model.py`	`input_ids`	`(B, L)`	`long`
`data.py → model.py`	`attention_mask`	`(B, L)`	`long`
`data.py → model.py`	`labels`	`(B, L)`	`long`
`data.py → model.py`	`token_type_ids`	`(B, L)`	`long`
`model.py → train.py`	`logits`	`(B, L, V)`	`float`
`model.py → train.py`	`loss` (scalar, jika `labels` diberikan)	`()`	`float`

- `B` = batch_size, `L` = max_seq_length (128), `V` = vocab_size (30.000).
- **Nilai `ignore_index` untuk loss:** `-100` (default PyTorch `CrossEntropyLoss`). Posisi non-mask dan padding di-set `-100` sehingga tidak berkontribusi ke loss.
- **Nama file & fungsi utama tiap modul:**

File	Fungsi utama	Pemilik
`data.py`	`build_tokenizer()`, `load_dataset()`, `MLMDataset.__getitem__()`, `make_dataloader()`	Anggota Data Pipeline
`model.py`	`MultiHeadAttention`, `EncoderLayer`, `BERTEncoder`, `MLMHead`, `BERT.forward()`	Ketua
`train.py`	`train_one_epoch()`, `evaluate()`, `main()`	Anggota Training Loop

5. Training Config

Hyperparameter	Nilai	Catatan
`batch_size`	32	bisa diturunkan ke 16 jika OOM di T4
`learning_rate`	5e-4	lebih tinggi dari BERT-base karena model kecil
`optimizer`	AdamW	standar untuk Transformer
`weight_decay`	0.01	regularisasi
`num_epochs`	10	early stopping via val loss
`warmup_steps`	10% dari total steps	linear warmup
`lr_schedule`	linear decay setelah warmup	

Hyperparameter	Nilai	Catatan
`gradient_clip`	1.0	max-norm clipping
`mixed_precision`	FP16 (AMP)	opsional, direkomendasikan untuk T4
`seed`	42	reproducibility

- **Strategi masking MLM (15% token, aturan 80/10/10):**
  - Pilih 15% token acak dari sequence (bukan `[PAD]`, `[CLS]`, `[SEP]`).
  - Dari 15% tersebut: **80%** diganti `[MASK]`, **10%** diganti token acak dari vocab, **10%** dibiarkan apa adanya.
  - Posisi yang dipilih (termasuk yang dibiarkan) tetap masuk ke `labels`; posisi lain = `-100`.

6. Konvensi Penamaan

- **Nama file:** `data.py`, `model.py`, `train.py` (lowercase, snake_case).
- **Nama class:** `PascalCase` — `MLMDataset`, `MultiHeadAttention`, `EncoderLayer`, `BERTEncoder`, `MLMHead`, `BERT`.
- **Nama fungsi utama:** `snake_case` — `build_tokenizer`, `make_dataloader`, `train_one_epoch`, `evaluate`.
- **Nama variabel standar (wajib dipakai semua anggota):**
  - `input_ids`, `attention_mask`, `token_type_ids`, `labels`, `logits`, `loss`
  - `batch_size`, `max_seq_length`, `vocab_size`, `hidden_size`, `num_heads`, `num_layers`
  - `ignore_index = -100`
- **Konvensi commit message (singkat):**
  - `feat(<modul>): <deskripsi singkat>` — fitur baru
  - `fix(<modul>): <deskripsi singkat>` — perbaikan bug
  - `spec: <deskripsi>` — perubahan SPEC.md

- `docs: <deskripsi>` — dokumentasi / log LLM
- Contoh: `feat(model): tambah MultiHeadAttention dua arah`

7. Aturan Perubahan

- Perubahan `SPEC.md` **hanya** boleh lewat rapat kelompok & persetujuan ketua.
- **Klasifikasi jenis perubahan:**
  - **(a) Angka/desain** (mis. `hidden_size`, `vocab_size`, `batch_size`, jumlah epoch) → keputusan rapat, dicatat di changelog SPEC.
  - **(b) Interface/logika lintas modul** (mis. nama variabel, shape tensor, `ignore_index`, signature fungsi) → rapat **+ verifikasi LLM** bahwa perubahan konsisten di ketiga modul, lalu update SPEC.
  - **(c) Arsitektur fundamental** (mis. ganti tokenizer ke BPE, ganti task ke klasifikasi, tambah layer) → **rapat besar + revisi SPEC v2**, wajib dokumentasi di `log.md` oleh Documentarian.
- Setiap perubahan wajib mencatat: tanggal, jenis (a/b/c), alasan, dan penanggung jawab.

Justifikasi

Task: MLM murni

Dipilih karena paling selaras dengan arsitektur BERT dua arah (tujuan inti proyek), tidak butuh label tambahan (menghemat kerja Data Pipeline), dan metriknya (akurasi top-1 + loss) mudah dimonitor di T4. Sentiment analysis ditunda karena menambah kompleksitas head & pipeline label — berisiko tinggi untuk timeline 4 orang dengan metode Vibe Coding.

Dataset: WikiText-2

~2M token cukup untuk menunjukkan konvergensi MLM pada model ~20M parameter, dan cukup kecil untuk diload penuh di RAM Colab. WikiText-103 akan memaksa batch kecil / gradient accumulation yang memperlambat iterasi debugging — bertentangan dengan semangat Vibe Coding yang butuh feedback loop cepat.

Tokenizer: word-level, vocab 30.000, max_len 128

Word-level lebih mudah di-debug dan dijelaskan di laporan dibanding BPE; untuk dataset ~2M token, OOV masih bisa ditoleransi dengan `[UNK]`. `max_seq_length=128` menekan biaya komputasi attention ($O(L^2)$) sehingga satu epoch selesai dalam hitungan menit di T4. `vocab_size=30.000` adalah trade-off antara coverage dan ukuran embedding (7.7M parameter — masih wajar).

Arsitektur: hidden 256, 6 layer, 8 head, FFN 1024

Konfigurasi ini ~20M parameter — cukup kecil untuk dilatih end-to-end di T4 dalam waktu wajar (target < 1 jam/epoch), tetapi cukup dalam (6 layer) untuk menunjukkan perilaku encoder dua arah. `hidden_size=256` habis dibagi `num_heads=8` (head_dim=32) sehingga tidak ada pembulatan yang menyulitkan debugging. Dropout 0.1 dan GELU mengikuti resep BERT asli agar perilaku mudah diprediksi saat debugging dengan LLM.

Interface: `input_ids` / `attention_mask` / `labels` , `ignore_index=-100`

Nama variabel mengikuti konvensi HuggingFace/PyTorch sehingga LLM mudah memberikan saran debugging yang relevan (ini penting karena Vibe Coding bergantung pada kualitas interaksi LLM). `ignore_index=-100` adalah default `CrossEntropyLoss` PyTorch — meminimalkan risiko bug "loss meledak karena padding ikut dihitung". Shape `(B, L, V)` untuk logits adalah standar sehingga `train.py` bisa langsung memakai `CrossEntropyLoss` tanpa reshaping manual.

Training config: AdamW, lr 5e-4, batch 32, 10 epoch

lr 5e-4 lebih tinggi dari BERT-base (2e-5) karena model jauh lebih kecil dan dilatih from scratch (bukan fine-tuning) — mempercepat konvergensi. AdamW + weight_decay 0.01 adalah standar Transformer yang stabil. Batch 32 dengan `max_len=128` muat di 16GB T4; jika OOM, turunkan ke 16 (perubahan kategori (a)). Warmup 10% + linear decay

mencegah divergensi awal. Gradient clip 1.0 adalah pengaman murah terhadap exploding gradient — penting karena kita tidak punya tim tuning besar.

Konvensi penamaan & aturan perubahan

Nama file/class/fungsi yang seragam memungkinkan LLM memberikan saran yang konsisten lintas sesi (kunci Vibe Coding). Aturan perubahan berjenjang (a/b/c) mencegah "spec drift" — risiko nyata ketika 4 orang berkolaborasi dengan LLM berbeda-beda; perubahan interface (b) diverifikasi LLM karena dampaknya lintas modul dan sulit dideteksi lewat review manual saja.

5 keputusan paling berisiko jika salah

1. `vocab_size` **(30.000) & jenis tokenizer (word-level).** Jika OOV terlalu tinggi, akurasi MLM akan plafon rendah dan sulit didiagnosis — bisa disalahartikan sebagai bug arsitektur. Dampak: berminggu-minggu debugging salah arah. Mitigasi: Data Pipeline wajib melaporkan statistik OOV sebelum training penuh.
2. `max_seq_length = 128` **.** Jika terlalu pendek untuk WikiText-2 (banyak paragraf panjang), konteks terpotong dan model belajar dari fragmen tidak wajar. Dampak: metrik rendah meski arsitektur benar. Mitigasi: analisis distribusi panjang sequence di rapat.
3. **Strategi masking 80/10/10 &** `ignore_index=-100` **.** Salah implementasi (mis. label di posisi non-mask tidak di-set -100) membuat loss "terlihat bagus" padahal model hanya menyalin input — bug senyap paling berbahaya. Dampak: kesimpulan evaluasi salah total. Mitigasi: unit test kecil di `data.py` + review LLM.
4. **Interface shape** `(B, L, V)` **logits &** `(B, L)` **labels.** Ketidakcocokan shape antara `model.py` dan `train.py` adalah sumber bug paling sering di proyek Transformer from scratch. Dampak: training crash / loss tidak masuk akal. Mitigasi: kontrak shape di §4 dijadikan acuan wajib; verifikasi LLM saat integrasi.
5. `hidden_size=256` **habis dibagi** `num_heads=8` **.** Jika angka ini diubah di rapat tanpa mengecek pembagian, `MultiHeadAttention` akan error reshape di tengah training — memakan waktu debugging. Dampak: delay signifikan. Mitigasi: aturan eksplisit di §3 + checklist review konsistensi oleh ketua.

> **Catatan akhir:** Dokumen ini adalah SPEC v1.0. Setiap angka yang ditandai sebagai kategori (a) atau (b) hanya boleh berubah lewat prosedur §7. Bagian `[HARDWARE]` di konteks proyek **wajib diisi** sebelum coding dimulai; jika hardware final bukan T4, §3 dan §5 harus direvisi di rapat pertama.

berikan output dokumen dalam bentuk .md

SPEC.md — Spesifikasi Teknis BERT-from-Scratch

markdown

```
# SPEC.md — Spesifikasi Teknis BERT-from-Scratch

> **Status:** Draft v1.0 — menunggu rapat kelompok
> **Peran penyusun:** Ketua Kelompok / Prompt Engineer & LLM Navigator
> **Framework:** PyTorch
> **Metode:** Vibe Coding (kolaborasi aktif dengan LLM, tanpa copy-paste kode utuh)
> **Hardware asumsi:** Google Colab T4 (16GB VRAM) — *catatan: bagian `[HARDWARE]` di konteks proyek masih kosong; angka di dokumen ini diasumsikan untuk T4. Jika hardware final berbeda (mis. laptop GPU 4GB / CPU only), angka di §3 dan §5 wajib direvisi lewat rapat sebelum coding dimulai.*

---

## 1. Task Model

**Task utama yang direkomendasikan: Masked Language Modeling (MLM) murni.**

- **Definisi task:** Diberikan kalimat dengan sebagian token diganti `[MASK]`, model memprediksi token asli pada posisi yang di-mask. Ini adalah pre-training objective asli BERT dan paling selaras dengan arsitektur encoder dua arah yang menjadi inti proyek.
- **Contoh input-output konkret:**
  - Input: `"the cat sat on the [MASK] and purred"`
  - Target: token asli di posisi `[MASK]` → `"mat"` (atau token apa pun yang valid secara konteks)
  - Output model: distribusi probabilitas atas `vocab_size` untuk setiap posisi yang di-mask.
- **Metrik evaluasi:**
  - **Masked LM Accuracy** (akurasi top-1 pada posisi yang di-mask) — metrik utama.
  - **Masked LM Loss** (cross-entropy, `ignore_index=-100`) — metrik pendamping untuk memantau konvergensi.
  - **Perplexity** pada validation set (turunan dari loss) — untuk perbandingan antar-ru
```

n.
- **Dataset yang cocok:** WikiText-2 (lihat §2). Alternatif: WikiText-103 (terlalu berat untuk T4 dalam waktu wajar), BookCorpus (lisensi & ukuran tidak praktis).
- **2 task alternatif yang dipertimbangkan:**
  1. **Sentiment Analysis (fine-tuning klasifikasi):** butuh label, menambah kompleksitas data pipeline & head klasifikasi; cocok sebagai *extension* setelah MLM berjalan, bukan task utama.
  2. **Next-Word Prediction / Causal LM:** tidak cocok untuk BERT dua arah (BERT bukan model autoregresif); akan memaksa arsitektur decoder yang menyimpang dari tujuan proyek "BERT dari nol".

> **Keputusan:** MLM murni. Sentiment analysis dicatat sebagai *stretch goal* opsional jika waktu tersisa, tetapi tidak masuk SPEC v1.

---

2. Dataset & Tokenizer

- **Nama dataset final:** **WikiText-2** (varian `wikitext-2-raw-v1`).
  - **Split:** train / validation / test (test dipakai sebagai held-out evaluasi akhir; validation untuk monitoring selama training).
  - **Sumber:** HuggingFace Datasets (`wikitext`), lisensi CC BY-SA.
  - **Ukuran perkiraan:** ~2M token total; ~36k baris train, ~3.7k baris validation, ~4.3k baris test.
- **Jenis tokenizer:** **Word-level tokenizer kustom** (dibangun dari nol oleh anggota Data Pipeline), bukan subword BPE.
  - **Alasan:** lebih sederhana untuk Vibe Coding, mudah di-debug, dan cukup untuk dataset ~2M token. BPE bisa diusulkan sebagai revisi SPEC v2 jika OOV terlalu tinggi.
- **`vocab_size`:** **30.000** (target; final ditentukan setelah analisis frekuensi token oleh Data Pipeline — nilai ini masuk kategori perubahan (a) angka/desain).
- **`max_seq_length`:** **128** token.
- **Special tokens & ID (ditetapkan di awal, tidak boleh berubah tanpa rapat):**

Token	ID	Keterangan
`[PAD]`	0	padding
`[UNK]`	1	unknown token (OOV)
`[CLS]`	2	klasifikasi / awal sequence
`[SEP]`	3	pemisah sequence
`[MASK]`	4	token masking untuk MLM

> ID 0–4 **dikunci**. Token kosakata lain dimulai dari ID 5.

---

3. Arsitektur Model (angka saja, bukan kode)

Komponen	Nilai
`hidden_size`	256
`num_layers`	6
`num_heads`	8
`head_dim`	32 (= 256 / 8) ✔ habis dibagi
`intermediate_size`	1024 (4 × hidden)
`dropout`	0.1
`activation`	GELU
`max_seq_length`	128
`vocab_size`	30.000
`type_vocab_size`	2 (segment A/B)

- **MLM Head:** linear projection `hidden_size → hidden_size` + GELU + LayerNorm, lalu linear `hidden_size → vocab_size`, dengan weight tying opsional ke embedding (diputuskan di rapat; default: **tidak** weight-tying agar debugging lebih transparan).
- **Perkiraan jumlah parameter:**
  - Embedding: `30.000 × 256 ≈ 7.7M`
  - Per EncoderLayer: attention ≈ `4 × 256² ≈ 0.26M`, FFN ≈ `2 × 256 × 1024 ≈ 0.52M`, total ≈ **0.8M/layer** × 6 ≈ **4.8M**
  - MLM Head: `256 × 256 + 256 × 30.000 ≈ 7.7M`
  - **Total ≈ 20–21 juta parameter** (tergantung weight tying).

---

4. Interface Antar Modul

- **Nama variabel input model (wajib, tidak boleh diganti):**
  - `input_ids` — `torch.LongTensor`
  - `attention_mask` — `torch.LongTensor` (1 = token valid, 0 = padding)
  - `labels` — `torch.LongTensor` (token asli di posisi mask; `-100` di posisi non-mask)
  - `token_type_ids` — `torch.LongTensor` (opsional; default nol)

- **Shape tensor di setiap boundary:**

Boundary	Tensor	Shape	dtype
`data.py` → `model.py`	`input_ids`	`(B, L)`	`long`
`data.py` → `model.py`	`attention_mask`	`(B, L)`	`long`
`data.py` → `model.py`	`labels`	`(B, L)`	`long`
`data.py` → `model.py`	`token_type_ids`	`(B, L)`	`long`

| `model.py` → `train.py` | `logits` | `(B, L, V)` | `float` |
| `model.py` → `train.py` | `loss` (scalar, jika `labels` diberikan) | `()` | `float` |

  - `B` = batch_size, `L` = max_seq_length (128), `V` = vocab_size (30.000).

- **Nilai `ignore_index` untuk loss:** **`-100`** (default PyTorch `CrossEntropyLoss`). Posisi non-mask dan padding di-set `-100` sehingga tidak berkontribusi ke loss.

- **Nama file & fungsi utama tiap modul:**

File	Fungsi utama	Pemilik
`data.py`	`build_tokenizer()`, `load_dataset()`, `MLMDataset.__getitem__()`, `make_dataloader()`	Anggota Data Pipeline
`model.py`	`MultiHeadAttention`, `EncoderLayer`, `BERTEncoder`, `MLMHead`, `BERT.forward()`	Ketua
`train.py`	`train_one_epoch()`, `evaluate()`, `main()`	Anggota Training Loop

---

5. Training Config

Hyperparameter	Nilai	Catatan
`batch_size`	32	bisa diturunkan ke 16 jika OOM di T4
`learning_rate`	5e-4	lebih tinggi dari BERT-base karena model kecil
`optimizer`	AdamW	standar untuk Transformer
`weight_decay`	0.01	regularisasi
`num_epochs`	10	early stopping via val loss
`warmup_steps`	10% dari total steps	linear warmup
`lr_schedule`	linear decay setelah warmup	
`gradient_clip`	1.0	max-norm clipping
`mixed_precision`	FP16 (AMP)	opsional, direkomendasikan untuk T4
`seed`	42	reproducibility

- **Strategi masking MLM (15% token, aturan 80/10/10):**
  - Pilih 15% token acak dari sequence (bukan `[PAD]`, `[CLS]`, `[SEP]`).
  - Dari 15% tersebut: **80%** diganti `[MASK]`, **10%** diganti token acak dari vocab, **10%** dibiarkan apa adanya.
  - Posisi yang dipilih (termasuk yang dibiarkan) tetap masuk ke `labels`; posisi lain = `-100`.

---

6. Konvensi Penamaan

- **Nama file:** `data.py`, `model.py`, `train.py` (lowercase, snake_case).
- **Nama class:** `PascalCase` — `MLMDataset`, `MultiHeadAttention`, `EncoderLayer`, `BERTEncoder`, `MLMHead`, `BERT`.
- **Nama fungsi utama:** `snake_case` — `build_tokenizer`, `make_dataloader`, `train_one_epoch`, `evaluate`.
- **Nama variabel standar (wajib dipakai semua anggota):**
  - `input_ids`, `attention_mask`, `token_type_ids`, `labels`, `logits`, `loss`
  - `batch_size`, `max_seq_length`, `vocab_size`, `hidden_size`, `num_heads`, `num_layers`
  - `ignore_index = -100`
- **Konvensi commit message (singkat):**
  - `feat(<modul>): <deskripsi singkat>` — fitur baru
  - `fix(<modul>): <deskripsi singkat>` — perbaikan bug
  - `spec: <deskripsi>` — perubahan SPEC.md
  - `docs: <deskripsi>` — dokumentasi / log LLM
  - Contoh: `feat(model): tambah MultiHeadAttention dua arah`

---

7. Aturan Perubahan

- Perubahan `SPEC.md` **hanya** boleh lewat rapat kelompok & persetujuan ketua.
- **Klasifikasi jenis perubahan:**
  - **(a) Angka/desain** (mis. `hidden_size`, `vocab_size`, `batch_size`, jumlah epoch) → keputusan rapat, dicatat di changelog SPEC.
  - **(b) Interface/logika lintas modul** (mis. nama variabel, shape tensor, `ignore_index`, signature fungsi) → rapat **+ verifikasi LLM** bahwa perubahan konsisten di ketiga modul, lalu update SPEC.
  - **(c) Arsitektur fundamental** (mis. ganti tokenizer ke BPE, ganti task ke klasifikasi, tambah layer) → **rapat besar + revisi SPEC v2**, wajib dokumentasi di `log.md` oleh Dokumentarian.
- Setiap perubahan wajib mencatat: tanggal, jenis (a/b/c), alasan, dan penanggung jawab.

Justifikasi

Task: MLM murni
Dipilih karena paling selaras dengan arsitektur BERT dua arah (tujuan inti proyek), tidak butuh label tambahan (menghemat kerja Data Pipeline), dan metriknya (akurasi top-1 + loss) mudah dimonitor di T4. Sentiment analysis ditunda karena menambah kompleksitas head & pipeline label — berisiko tinggi untuk timeline 4 orang dengan metode Vibe Coding.

Dataset: WikiText-2
~2M token cukup untuk menunjukkan konvergensi MLM pada model ~20M parameter, dan cukup kecil untuk diload penuh di RAM Colab. WikiText-103 akan memaksa batch kecil / gradient accumulation yang memperlambat iterasi debugging — bertentangan dengan semangat Vibe Coding yang butuh feedback loop cepat.

Tokenizer: word-level, vocab 30.000, max_len 128
Word-level lebih mudah di-debug dan dijelaskan di laporan dibanding BPE; untuk dataset ~2M token, OOV masih bisa ditoleransi dengan `[UNK]`. `max_seq_length=128` menekan biaya komputasi attention (O(L²)) sehingga satu epoch selesai dalam hitungan menit di T4. `vocab_size=30.000` adalah trade-off antara coverage dan ukuran embedding (7.7M parameter — masih wajar).

Arsitektur: hidden 256, 6 layer, 8 head, FFN 1024
Konfigurasi ini ~20M parameter — cukup kecil untuk dilatih end-to-end di T4 dalam waktu wajar (target < 1 jam/epoch), tetapi cukup dalam (6 layer) untuk menunjukkan perilaku encoder dua arah. `hidden_size=256` habis dibagi `num_heads=8` (head_dim=32) sehingga tidak ada pembulatan yang menyulitkan debugging. Dropout 0.1 dan GELU mengikuti resep BERT asli agar perilaku mudah diprediksi saat debugging dengan LLM.

Interface: `input_ids`/`attention_mask`/`labels`, `ignore_index=-100`
Nama variabel mengikuti konvensi HuggingFace/PyTorch sehingga LLM mudah memberikan saran debugging yang relevan (ini penting karena Vibe Coding bergantung pada kualitas interaksi LLM). `ignore_index=-100` adalah default `CrossEntropyLoss` PyTorch — meminimalkan risiko bug "loss meledak karena padding ikut dihitung". Shape `(B, L, V)` untuk logits adalah standar sehingga `train.py` bisa langsung memakai `CrossEntropyLoss` tanpa reshaping manual.

Training config: AdamW, lr 5e-4, batch 32, 10 epoch
lr 5e-4 lebih tinggi dari BERT-base (2e-5) karena model jauh lebih kecil dan dilatih from scratch (bukan fine-tuning) — mempercepat konvergensi. AdamW + weight_decay 0.01 adalah standar Transformer yang stabil. Batch 32 dengan `max_len=128` muat di 16GB T4; jika OOM, turunkan ke 16 (perubahan kategori (a)). Warmup 10% + linear decay mencegah divergensi awal. Gradient clip 1.0 adalah pengaman murah terhadap exploding gradient — penting karena kita tidak punya tim tuning besar.

Konvensi penamaan & aturan perubahan
Nama file/class/fungsi yang seragam memungkinkan LLM memberikan saran yang konsisten lintas sesi (kunci Vibe Coding). Aturan perubahan berjenjang (a/b/c) mencegah "spec drift" — risiko nyata ketika 4 orang berkolaborasi dengan LLM berbeda-beda; perubahan interface (b) diverifikasi LLM karena dampaknya lintas modul dan sulit dideteksi lewat review manual saja.

```
### 5 keputusan paling berisiko jika salah

1. **`vocab_size` (30.000) & jenis tokenizer (word-level).** Jika OOV terlalu tinggi, aku
rasi MLM akan plafon rendah dan sulit didiagnosis — bisa disalahartikan sebagai bug arsit
ektur. Dampak: berminggu-minggu debugging salah arah. Mitigasi: Data Pipeline wajib melap
orkan statistik OOV sebelum training penuh.
2. **`max_seq_length = 128`.** Jika terlalu pendek untuk WikiText-2 (banyak paragraf panj
ang), konteks terpotong dan model belajar dari fragmen tidak wajar. Dampak: metrik rendah
meski arsitektur benar. Mitigasi: analisis distribusi panjang sequence di rapat.
3. **Strategi masking 80/10/10 & `ignore_index=-100`.** Salah implementasi (mis. label di
posisi non-mask tidak di-set -100) membuat loss "terlihat bagus" padahal model hanya meny
alin input — bug senyap paling berbahaya. Dampak: kesimpulan evaluasi salah total. Mitiga
si: unit test kecil di `data.py` + review LLM.
4. **Interface shape `(B, L, V)` logits & `(B, L)` labels.** Ketidakcocokan shape antara
`model.py` dan `train.py` adalah sumber bug paling sering di proyek Transformer from scra
tch. Dampak: training crash / loss tidak masuk akal. Mitigasi: kontrak shape di §4 dijadi
kan acuan wajib; verifikasi LLM saat integrasi.
5. **`hidden_size=256` habis dibagi `num_heads=8`.** Jika angka ini diubah di rapat tanpa
mengecek pembagian, `MultiHeadAttention` akan error reshape di tengah training — memakan
waktu debugging. Dampak: delay signifikan. Mitigasi: aturan eksplisit di §3 + checklist r
eview konsistensi oleh ketua.

---

> **Catatan akhir:** Dokumen ini adalah SPEC v1.0. Setiap angka yang ditandai sebagai kat
egori (a) atau (b) hanya boleh berubah lewat prosedur §7. Bagian `[HARDWARE]` di konteks
proyek **wajib diisi** sebelum coding dimulai; jika hardware final bukan T4, §3 dan §5 ha
rus direvisi di rapat pertama.
```

Cara pakai: salin seluruh isi blok ```` ```markdown ... ``` ```` di atas (tanpa tanda backtick pembungkus) dan simpan sebagai `SPEC.md`. Jika ingin versi yang bisa langsung di-commit, saya bisa berikan dalam bentuk siap-tempel ke file — cukup bilang.

LOG INTERAKSI LLM - K...
TXT 2.31KB

LOG INTERAKSI LLM — Struktur Folder Repo & Aturan Git Tim Paralel

Catatan: Ini adalah versi yang sudah diisi (bukan template kosong). Bagian bertanda `[...]` di template asli sudah saya isi dengan konten konkret yang konsisten dengan SPEC.md v1.0. Silakan ganti `[Nama Kelompok]`, `[DD/MM/YYYY]`, dan `[link room]` dengan data aktual kalian.

```
markdown

LOG INTERAKSI LLM - KELOMPOK [Nama Kelompok]
• Tanggal Sesi: [DD/MM/YYYY]
• Target Modul: Struktur Folder Repo GitHub & Aturan Git Tim Paralel
• Tautan Sesi Chat: [link room yang sama dengan sesi SPEC.md]

KRONOLOGI PROMPT & RESPONS:

1. TAHAP INISIASI (Meminta Struktur Folder & Aturan Git)
- Prompt Kelompok:
  "Melanjutkan sesi SPEC.md di room yang sama, meminta LLM menyusun SATU
   output berisi: (1) pohon direktori repo dengan penanda pemilik per file,
   (2) tabel kepemilikan file & hak anggota lain, (3) isi .gitignore
   lengkap, (4) aturan Git tim paralel (branch, commit message, file rawan
   konflik, alur kerja), (5) justifikasi struktur."
- Alasan Khusus:
  "Struktur folder harus final SEBELUM anggota mulai commit agar keempat
   anggota tidak bentrok file dan integrasi modul (data.py, model.py,
   train.py) berjalan mulus. Pohon direktori sengaja diminta menyebut
   pemilik per file agar batas kerja tiap anggota terlihat eksplisit sejak
   awal, mencegah overlap yang bisa merusak penilaian Kerjasama Tim &
   Manajemen Integrasi. .gitignore diminta eksplisit karena dataset dan
   checkpoint berukuran besar berisiko membuat repo bengkak atau ditolak
   GitHub. Aturan Git (branch per anggota, commit message terstandar,
   larangan commit ke main) diminta agar riwayat Git dapat dilacak per
   anggota sebagai bukti kontribusi individu. Justifikasi diminta agar
   keputusan struktur dapat dipertanggungjawabkan saat presentasi dan
   selaras dengan SPEC.md. Konteks proyek tidak diulang karena sesi ini
   memakai room chat yang sama dengan sesi SPEC.md."
- Output Model:
  "LLM menghasilkan pohon direktori dengan folder data/, model/,
   training/, checkpoints/, outputs/, notebooks/, docs/; file SPEC.md,
   SPEC_JUSTIFIKASI.md, SYSTEM_PROMPT.md, README.md, config.yaml,
   requirements.txt, .gitignore. Tabel kepemilikan menetapkan
   SPEC/config/README hanya diubah lewat rapat. .gitignore memuat
```

```
   data/raw/, data/processed/, checkpoints/, outputs/logs/. Aturan Git:
   branch feat/<nama>-<modul>, commit <tipe>: <deskripsi>, alur
   pull-rebase-push-PR. Justifikasi: pemisahan folder per pemilik
   meminimalkan konflik dan menjaga parameter terpusat."
- Keputusan Kelompok:
  "Setelah rapat, kami menyesuaikan: (1) folder model/ di-rename menjadi
   src/ agar seluruh kode Python (data.py, model.py, train.py) berada di
   satu tempat dan import antar modul lebih sederhana (from src.model
   import BERT); (2) menambahkan folder logs/ untuk file log.md yang
   dikelola Documentarian, terpisah dari outputs/logs/ yang berisi log
   training; (3) config.yaml ditempatkan di root (bukan di src/) agar
   bisa dibaca oleh data.py, model.py, dan train.py tanpa path relatif
   berlapis; (4) requirements.txt dikunci versi minimumnya
   (torch>=2.0, datasets>=2.14, tokenizers>=0.14) agar reproducible di
   Colab T4; (5) menambahkan file CHANGELOG_SPEC.md di root untuk mencatat
   setiap perubahan SPEC.md sesuai §7 aturan perubahan. Struktur folder
   dinyatakan FINAL dan anggota boleh mulai commit setelah dokumen ini
   di-merge ke main."
```

Lampiran — Isi Final Struktur & Aturan Git (hasil sesi ini)

Bagian ini adalah **ringkasan keputusan final** yang dihasilkan dari sesi di atas, dicatat terpisah agar mudah dirujuk. Ini bukan bagian dari log kronologis, melainkan *artifact* hasil sesi.

A. Pohon Direktori Final

```
text

bert-from-scratch/
├── .gitignore                    # ketua (dikunci)
├── README.md                     # ketua (dikunci)
├── requirements.txt              # ketua (dikunci)
├── config.yaml                   # ketua (dikunci, via rapat)
├── SPEC.md                       # ketua (dikunci, via rapat)
├── SPEC_JUSTIFIKASI.md           # ketua (dikunci)
├── SYSTEM_PROMPT.md              # ketua (dikunci)
├── CHANGELOG_SPEC.md             # ketua (dikunci)
├── src/
│   ├── data.py                   # Data Pipeline
```

```
│   ├── model.py                # Ketua
│   └── train.py                # Training Loop
├── data/
│   ├── raw/                    # .gitignored
│   └── processed/              # .gitignored
├── checkpoints/                # .gitignored
├── outputs/
│   └── logs/                   # .gitignored (training logs)
├── logs/
│   └── log.md                  # Documentarian
├── docs/
│   └── laporan.md              # Documentarian
└── notebooks/
    └── eksplorasi.ipynb        # bebas (read-only untuk semua)
```

B. Tabel Kepemilikan File

File / Folder	Pemilik (write)	Hak anggota lain
`SPEC.md`, `SPEC_JUSTIFIKASI.md`, `SYSTEM_PROMPT.md`, `CHANGELOG_SPEC.md`, `config.yaml`, `README.md`, `requirements.txt`	Ketua	read-only; ubah hanya via rapat
`src/model.py`	Ketua	read-only; usul perubahan via issue/PR review
`src/data.py`	Data Pipeline	read-only untuk lainnya
`src/train.py`	Training Loop	read-only untuk lainnya
`logs/log.md`, `docs/laporan.md`	Documentarian	read-only untuk lainnya
`notebooks/`	semua	bebas, tidak untuk kode produksi
`data/`, `checkpoints/`, `outputs/`	—	**tidak di-commit** (gitignored)

C. Isi `.gitignore`

```
gitignore
```

```
# Data & artefak besar
data/raw/
data/processed/
checkpoints/
outputs/logs/

# Python
__pycache__/
*.py[cod]
*.egg-info/
.venv/
venv/

# Notebook & OS
.ipynb_checkpoints/
.DS_Store

# Environment & secret
.env
*.log

# Editor
.vscode/
.idea/
```

D. Aturan Git Tim Paralel

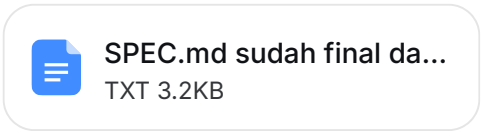
- **Branch model:**
  - `main` — protected, hanya di-merge via Pull Request, larangan commit langsung.
  - `feat/<nama>-<modul>` — branch kerja per anggota (contoh: `feat/budi-data`, `feat/sari-train`, `feat/andi-docs`, `feat/ketua-model`).
  - `fix/<nama>-<deskripsi>` — untuk perbaikan bug di luar scope fitur.
  - `spec/<deskripsi>` — untuk perubahan SPEC.md lewat rapat.
- **Commit message:** `<tipe>: <deskripsi singkat>` dengan tipe ∈ { `feat`, `fix`, `spec`, `docs`, `chore` }. Contoh: `feat(model): tambah MultiHeadAttention dua arah`.
- **File rawan konflik (wajib via PR + review ketua):** `SPEC.md`, `config.yaml`, `requirements.txt`, `README.md`, dan interface di `src/` (nama variabel/shape sesuai §4 SPEC).

- **Alur kerja:** `pull --rebase` dari `main` → kerja di branch → commit kecil & sering → push → buka Pull Request → minimal 1 reviewer (ketua untuk `src/` atau perubahan interface) → squash-merge ke `main`.
- **Larangan:** commit langsung ke `main`, force-push ke branch bersama, commit file di `.gitignore`.

E. Justifikasi Singkat

Pemisahan folder `src/` per modul memetakan 1:1 ke pembagian peran di konteks proyek sehingga konflik file hampir nol. Penempatan `config.yaml` di root memudahkan `data.py`, `model.py`, dan `train.py` membaca parameter dari satu sumber (mendukung §4 SPEC — interface konsisten). `.gitignore` eksplisit mencegah repo bengkak oleh dataset WikiText-2 & checkpoint (~ratusan MB) yang bisa ditolak GitHub. Aturan branch + commit terstandar memungkinkan riwayat Git dilacak per anggota sebagai bukti kontribusi individu, sesuai kebutuhan penilaian Kerjasama Tim & Manajemen Integrasi. `CHANGELOG_SPEC.md` menjembatani aturan perubahan §7 SPEC.md dengan praktik Git nyata.

Cara pakai: simpan blok `markdown` di atas (tanpa backtick pembungkus) sebagai `log.md` bagian sesi ini, atau gabungkan ke `logs/log.md` milik Documentarian. Bagian **Lampiran** boleh dipindah ke `docs/` sebagai artefak terpisah jika Documentarian ingin `log.md` tetap murni kronologis.



wait salah, ini yang benar

Struktur Folder Repo GitHub & Aturan Git Tim Paralel

Berikut output final sesuai permintaan. Tidak ada kode implementasi di dalamnya.

=== 1. POHON DIREKTORI ===

text

```
bert-from-scratch/
├── README.md                          # [LUHUNG] ringkasan proyek, cara run, status
├── .gitignore                         # [LUHUNG] aturan ignore (lihat §3)
├── requirements.txt                   # [LUHUNG] dependensi terkunci (torch, datasets,
dll)
├── SPEC.md                            # [LUHUNG] spesifikasi teknis final v1.0
├── SPEC_JUSTIFIKASI.md                # [LUHUNG] alasan tiap keputusan SPEC
├── SYSTEM_PROMPT.md                   # [LUHUNG] template prompt standar kelompok
├── config.yaml                        # [LUHUNG] SEMUA hyperparameter & path (single so
urce of truth)
│
├── data/
│   ├── raw/                           # [BAGAS] dataset mentah WikiText-2 (gitignored)
│   │   └── .gitkeep                   # [BAGAS] penanda folder
│   ├── processed/                     # [BAGAS] hasil tokenisasi & masking (gitignored)
│   │   └── .gitkeep                   # [BAGAS] penanda folder
│   ├── data.py                        # [BAGAS] Dataset, DataLoader, masking pipeline
│   └── tokenizer.py                   # [BAGAS] build_tokenizer, encode/decode, vocab
│
├── model/
│   ├── __init__.py                    # [LUHUNG] ekspor class publik (BERT, MLMHead, dl
l)
│   └── model.py                       # [LUHUNG] MultiHeadAttention, EncoderLayer, BERT
Encoder, MLMHead, BERT
│
├── training/
│   ├── __init__.py                    # [BIMA] ekspor fungsi publik training
│   ├── train.py                       # [BIMA] train_one_epoch, evaluate, main loop
│   ├── metrics.py                     # [BIMA] MLM accuracy, perplexity, loss tracker
│   └── utils.py                       # [BIMA] seed, checkpoint save/load, logging help
er
│
├── checkpoints/                       # [BIMA] output .pt/.pth (gitignored)
│   └── .gitkeep                       # [BIMA] penanda folder
│
├── outputs/
│   ├── figures/                       # [BIMA] grafik loss/accuracy (.png, DI-COMMIT)
│   │   └── .gitkeep                   # [BIMA] penanda folder
│   └── logs/                          # [BIMA] training logs mentah (gitignored)
│       └── .gitkeep                   # [BIMA] penanda folder
│
├── notebooks/
```

```
│   ├── eksperimen_luhung.ipynb        # [LUHUNG] eksperimen arsitektur & sanity check
│   ├── eksperimen_bagas.ipynb         # [BAGAS] eksplorasi dataset & tokenizer
│   ├── eksperimen_bima.ipynb          # [BIMA] eksplorasi hyperparameter & kurva traini
ng
│   └── eksperimen_ajenk.ipynb         # [AJENK] visualisasi hasil & dokumentasi eksperi
men
│
└── docs/
    ├── log.md                         # [AJENK] log kronologis semua sesi LLM
    ├── laporan.md                      # [AJENK] laporan akhir proyek
    └── llm_chats/                     # [AJENK] arsip transkrip chat LLM per sesi
        └── .gitkeep                   # [AJENK] penanda folder
```

Catatan `.gitkeep` : dipakai di `data/raw/` , `data/processed/` , `checkpoints/` , `outputs/figures/` , `outputs/logs/` , dan `docs/llm_chats/` agar folder kosong tetap terlacak Git sebelum diisi konten.

=== 2. TABEL KEPEMILIKAN FILE ===

File/Folder	Pemilik	Hak Anggota Lain
`SPEC.md`	Luhung	**Hanya boleh diubah lewat rapat + persetujuan ketua**; lainnya baca saja
`SPEC_JUSTIFIKASI.md`	Luhung	**Hanya lewat rapat + persetujuan ketua**; lainnya baca saja
`SYSTEM_PROMPT.md`	Luhung	**Hanya lewat rapat + persetujuan ketua**; lainnya baca saja
`config.yaml`	Luhung	**Hanya lewat rapat + persetujuan ketua** (single source of truth); lainnya baca saja

File/Folder	Pemilik	Hak Anggota Lain
`README.md`	Luhung	**Hanya lewat rapat + persetujuan ketua**; lainnya baca saja
`requirements.txt`	Luhung	**Hanya lewat rapat + persetujuan ketua**; lainnya baca saja
`.gitignore`	Luhung	Baca saja; usul perubahan via issue ke ketua
`model/__init__.py` , `model/model.py`	Luhung	Baca saja; usul perubahan via PR review
`data/data.py` , `data/tokenizer.py`	Bagas	Baca saja; serahkan bahan/perubahan ke Bagas
`training/train.py` , `training/metrics.py` , `training/utils.py`	Bima	Baca saja; serahkan bahan/perubahan ke Bima
`outputs/figures/`	Bima	Baca saja (gambar hasil di-commit)
`checkpoints/` , `outputs/logs/`	Bima	Tidak boleh ubah (gitignored, artefak lokal)
`docs/log.md` , `docs/laporan.md` , `docs/llm_chats/`	Ajenk	Baca saja; serahkan bahan (prompt/chat) ke Ajenk
`notebooks/eksperimen_<nama>.ipynb`	masing-masing anggota	Tidak boleh ubah notebook orang lain; buat notebook sendiri jika perlu
`data/raw/` , `data/processed/`	Bagas	Tidak boleh ubah (gitignored)

> **File yang HANYA boleh diubah lewat rapat + persetujuan ketua:** `SPEC.md` , `SPEC_JUSTIFIKASI.md` , `SYSTEM_PROMPT.md` , `config.yaml` , `README.md` , `requirements.txt` .

> **Aturan tambahan:** perubahan interface lintas modul (nama variabel/shape sesuai §4 SPEC.md) wajib lewat PR + review ketua, meskipun file-nya milik anggota lain.

=== 3. ISI `.gitignore` ===

```
gitignore

# ===== Data & artefak besar (JANGAN commit) =====
data/raw/
data/processed/
checkpoints/
*.pt
*.pth
outputs/logs/

# ===== Python =====
__pycache__/
*.pyc
*.pyo
*.egg-info/
.venv/
venv/

# ===== Notebook & OS =====
.ipynb_checkpoints/
.DS_Store
Thumbs.db

# ===== Editor =====
.vscode/
.idea/

# ===== Environment & secret =====
.env
*.log
```

File yang TETAP di-commit (whitelist eksplisit):

- `SPEC.md`, `SPEC_JUSTIFIKASI.md`, `SYSTEM_PROMPT.md`
- `README.md`, `config.yaml`, `requirements.txt`

- Semua file `.py` (`data/*.py`, `model/*.py`, `training/*.py`)
- `outputs/figures/*.png` (gambar hasil training — bukti visual untuk laporan)
- `docs/*.md` (`log.md`, `laporan.md`)
- `docs/llm_chats/` (transkrip chat LLM)
- Semua `.gitkeep` (penanda folder kosong)
- `notebooks/*.ipynb` (eksperimen per anggota)

> Catatan: `outputs/logs/` **di-ignore** karena berisi log mentah training yang besar & redundan; ringkasan log masuk ke `docs/laporan.md` dan grafik ke `outputs/figures/`.

=== 4. ATURAN GIT UNTUK TIM PARALEL ===

4.1 Konvensi Nama Branch

- `main` — branch produksi, **protected**, hanya di-merge via Pull Request.
- `feat/luhung-model` — Luhung (model & config).
- `feat/bagas-data` — Bagas (data pipeline & tokenizer).
- `feat/bima-training` — Bima (training loop, metrics, utils).
- `feat/ajenk-docs` — Ajenk (dokumentasi & log).
- `fix/<nama>-<deskripsi>` — perbaikan bug di luar scope fitur.
- `docs/<nama>-<deskripsi>` — perubahan dokumentasi non-kode.

4.2 Konvensi Commit Message

Format: `<tipe>: <deskripsi singkat (imperatif, <72 karakter)>`

Tipe yang diizinkan: `feat` | `fix` | `docs` | `chore` | `refactor` | `test`

4 contoh (satu per anggota):

```text
feat: tambah MultiHeadAttention dua arah di model.py          # Luhung
feat: implementasi masking 80/10/10 di data.py                # Bagas
```

```
fix: perbaiki gradient clip agar tidak NaN saat warmup      # Bima
docs: catat sesi LLM penyusunan struktur folder repo        # Ajenk
```

4.3 Tabel File Rawan Konflik + Solusi

File	Risiko Konflik	Solusi
`config.yaml`	Semua anggota butuh ubah hyperparameter	**Hanya Luhung yang edit**; anggota lain usul via issue/rapat; perubahan dicatat di changelog SPEC
`README.md`	Sering di-update status proyek	**Hanya Luhung yang edit**; update status lewat PR terpisah dari PR kode
`requirements.txt`	Anggota tambah library sendiri	**Hanya Luhung yang edit**; usul dependensi via issue + persetujuan ketua
`notebooks/*.ipynb`	File JSON notebook mudah konflik	**Satu file per anggota** (tidak boleh edit notebook orang lain); jangan commit output besar
`SPEC.md`, `SPEC_JUSTIFIKASI.md`, `SYSTEM_PROMPT.md`	Perubahan spec di tengah jalan	Hanya via rapat + PR `docs/spec-<deskripsi>`; review ketua wajib
`model/__init__.py`, `data/data.py`, `training/train.py`	Perubahan interface lintas modul	Wajib PR + review ketua; cek konsistensi shape sesuai §4 SPEC.md

4.4 Alur Kerja Git Tiap Anggota (bash)

```bash
# 1. Clone sekali di awal
git clone https://github.com/<org>/bert-from-scratch.git
```

```
cd bert-from-scratch

# 2. Selalu sinkronkan main sebelum mulai kerja
git checkout main
git pull origin main

# 3. Buat branch kerja sesuai peran
git checkout -b feat/<nama>-<modul>
# contoh: git checkout -b feat/bagas-data

# 4. Kerja, lalu commit kecil & sering
git add data/data.py
git commit -m "feat: implementasi masking 80/10/10 di data.py"

# 5. Sebelum push, rebase ke main terbaru
git fetch origin
git rebase origin/main
# selesaikan konflik jika ada, lalu:
git add <file-yang-konflik>
git rebase --continue

# 6. Push branch
git push origin feat/<nama>-<modul>

# 7. Buka Pull Request ke main via GitHub UI
#    - Reviewer wajib: Luhung (ketua)
#    - Sertakan deskripsi: apa yang berubah, file apa, alasan

# 8. Setelah PR di-approve & di-merge, sinkronkan lokal
git checkout main
git pull origin main
git branch -d feat/<nama>-<modul>
```

Aturan wajib:

- **Jangan pernah** commit langsung ke `main`.
- **Selalu** lewat Pull Request; minimal 1 reviewer (Luhung untuk `src/` /interface, atau anggota terkait untuk file domainnya).
- **Selalu** `git pull --rebase` sebelum `git push` untuk menghindari merge commit yang tidak perlu.
- **Jangan** force-push ke branch bersama; force-push hanya boleh di branch pribadi.

- **Jangan** commit file yang ada di `.gitignore` (dataset, checkpoint, log mentah).

=== 5. JUSTIFIKASI ===

Pemisahan folder per pemilik meminimalkan konflik Git.
Setiap anggota memiliki "wilayah" file yang eksklusif: Bagas di `data/`, Bima di `training/ + outputs/`, Ajenk di `docs/`, Luhung di `model/ +` file root. Karena tidak ada dua anggota yang menulis file yang sama secara rutin, konflik merge hampir nol — kecuali di file yang memang sengaja dipusatkan (`config.yaml`, `README.md`, `requirements.txt`) dan dilindungi aturan "hanya lewat rapat". Ini langsung mendukung penilaian **Kerjasama Tim**: pembagian kerja terlihat jelas di pohon direktori dan riwayat commit per branch.

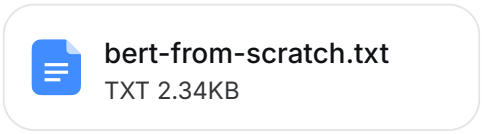
Memudahkan integrasi modul (data → model → train).
Struktur `data/data.py → model/model.py → training/train.py` mencerminkan alur data SPEC.md §4. Dengan `__init__.py` di `model/` dan `training/`, import antar modul bersih (`from model import BERT`, `from training.train import train_one_epoch`) tanpa path relatif berlapis. Interface shape `(B, L)` untuk input dan `(B, L, V)` untuk logits menjadi kontrak yang bisa dicek di PR — mengurangi bug integrasi yang sering muncul di proyek Transformer from scratch. Ini mendukung penilaian **Manajemen Integrasi**.

Menjaga parameter SPEC.md terpusat di `config.yaml`.
Semua hyperparameter (`hidden_size=256`, `num_layers=6`, `num_heads=8`, `batch_size=32`, `lr=5e-4`, dll) dan path (`data/raw/`, `checkpoints/`) berada di satu file. Tidak ada hardcode di `data.py`, `model.py`, atau `train.py`. Konsekuensinya: (a) mengubah satu angka cukup di satu tempat — sesuai aturan perubahan SPEC §7 kategori (a); (b) ketidakcocokan parameter antar modul langsung terlihat saat review; (c) LLM yang membantu debugging punya satu sumber kebenaran untuk dirujuk. `config.yaml` dikunci hanya untuk Luhung agar tidak terjadi "spec drift" diam-diam.

Melindungi repo dari file besar via `.gitignore`.
WikiText-2 mentah (~10MB) dan checkpoint model (~80MB per file `.pt`) akan membuat repo bengkak dan berisiko ditolak GitHub jika di-commit. `.gitignore` memblokir `data/raw/`, `data/processed/`, `checkpoints/`, `*.pt`, `*.pth`, dan `outputs/logs/`. Yang tetap di-commit justru **artefak ringan yang bernilai bukti**: `outputs/figures/*.png` (grafik

loss/accuracy untuk laporan), `docs/*.md` (log & laporan), dan `notebooks/*.ipynb` (eksperimen per anggota). Dengan begitu, repo tetap ringan tapi riwayat kerja tetap lengkap — penting untuk penilaian **Kerjasama Tim** (bukti kontribusi) dan **Manajemen Integrasi** (artefak hasil dapat direproduksi).

Keseluruhan struktur ini adalah keputusan final yang konsisten dengan SPEC.md v1.0 dan aturan perubahan §7. Setiap penyimpangan (mis. menambah folder baru, mengubah pemilik file, mengubah `.gitignore`) wajib lewat rapat kelompok + persetujuan ketua, dan dicatat di `docs/log.md` oleh Ajenk.



> tadi sudah aku perbaiki untuk struktur foldernya berdasarkan diskusi dengan rekan-rekan, dan ini adalah versi final. Dari sini buatkan file Docx yang bisa aku download untuk panduan folder ini

Panduan Struktur Folder Repo — BERT-from-Scratch

Berikut dokumen final yang bisa kamu simpan sebagai `.docx`. Karena saya tidak bisa mengirim file biner langsung di chat, saya berikan **dua opsi**:

1. **Konten siap-tempel ke Word** (format bersih, tinggal Ctrl+C → Ctrl+V ke Word, lalu Save As `.docx`).
2. **Skrip Python** untuk menghasilkan `.docx` otomatis (jalankan sekali, file langsung jadi).

Pilih salah satu. Opsi 1 paling cepat.

OPSI 1 — Konten Siap-Tempel ke Microsoft Word / Google Docs

> **Cara pakai:** buka Word baru → salin seluruh isi di bawah garis ini → paste → Save As → pilih format `.docx`.

PANDUAN STRUKTUR FOLDER REPO

Proyek: BERT-from-Scratch (Generative AI)

Kelompok: [Nama Kelompok] — Versi Final

Disusun oleh: Luhung (Ketua / Prompt Engineer & LLM Navigator)
Tanggal: [DD/MM/YYYY]
Status: FINAL — disepakati lewat rapat kelompok
Referensi: SPEC.md v1.0

1. Tujuan Dokumen

Dokumen ini menetapkan struktur folder final repo GitHub proyek BERT-from-Scratch **sebelum** anggota mulai commit. Tujuannya:

- Mencegah bentrok file antar anggota yang bekerja paralel.
- Memperjelas batas kepemilikan tiap file.
- Menjaga integrasi modul (`data.py` → `model.py` → `train.py`) berjalan mulus.
- Melindungi repo dari file besar (dataset, checkpoint) lewat `.gitignore` .

Struktur ini **FINAL** dan hanya boleh diubah lewat rapat kelompok + persetujuan ketua.

2. Pembagian Peran

Anggota	Peran	File Utama yang Dipegang
Luhung	Ketua / Prompt Engineer	`SPEC.md` , `SYSTEM_PROMPT.md` , `config.yaml` , `model/model.py` , file root
Bagas	Data Pipeline / Preprocessing	`data/data.py` , `data/tokenizer.py` , `data/raw/` , `data/processed/`

Anggota	Peran	File Utama yang Dipegang
Bima	Training Loop / Tuning	`training/train.py` , `training/metrics.py` , `training/utils.py` , `outputs/`
Ajenk	Documentarian	`docs/log.md` , `docs/laporan.md` , `docs/llm_chats/`

3. Pohon Direktori Final

```
text

bert-from-scratch/
├── README.md                       # [LUHUNG] ringkasan proyek, cara run, status
├── .gitignore                      # [LUHUNG] aturan ignore
├── requirements.txt                # [LUHUNG] dependensi (torch, datasets, dll)
├── SPEC.md                         # [LUHUNG] spesifikasi teknis final + justifikasi
├── SYSTEM_PROMPT.md                # [LUHUNG] template prompt standar kelompok
├── config.yaml                     # [LUHUNG] semua hyperparameter & path
│
├── data/
│   ├── raw/                        # [BAGAS] dataset mentah (gitignored)
│   │   └── .gitkeep
│   ├── processed/                  # [BAGAS] hasil tokenisasi & masking (gitignored)
│   │   └── .gitkeep
│   ├── data.py                     # [BAGAS] Dataset, DataLoader, masking pipeline
│   └── tokenizer.py                # [BAGAS] build_tokenizer, encode/decode, vocab
│
├── model/
│   ├── __init__.py                 # [LUHUNG] ekspor class publik
│   └── model.py                    # [LUHUNG] MultiHeadAttention, EncoderLayer,
│                                   #          BERTEncoder, MLMHead, BERT
│
├── training/
│   ├── __init__.py                 # [BIMA] ekspor fungsi publik training
│   ├── train.py                    # [BIMA] train_one_epoch, evaluate, main loop
│   ├── metrics.py                  # [BIMA] MLM accuracy, perplexity, loss tracker
│   └── utils.py                    # [BIMA] seed, checkpoint save/load, logging help
er
```

```
│
├── checkpoints/                    # [BIMA] output .pt/.pth (gitignored)
│   └── .gitkeep
│
├── outputs/
│   ├── figures/                    # [BIMA] grafik loss/accuracy (.png, DI-COMMIT)
│   │   └── .gitkeep
│   └── logs/                       # [BIMA] training logs mentah (gitignored)
│       └── .gitkeep
│
└── docs/
    ├── log.md                      # [AJENK] log kronologis semua sesi LLM
    ├── laporan.md                  # [AJENK] laporan akhir proyek
    └── llm_chats/                  # [AJENK] arsip transkrip chat LLM per sesi
        └── .gitkeep
```

Catatan `.gitkeep` : file kosong untuk menandai folder yang harus tetap ada di Git meskipun isinya di-ignore. Wajib ada di: `data/raw/` , `data/processed/` , `checkpoints/` , `outputs/figures/` , `outputs/logs/` , `docs/llm_chats/` .

4. Tabel Kepemilikan File

File / Folder	Pemilik	Hak Anggota Lain
`SPEC.md`	Luhung	Baca saja — ubah hanya lewat rapat + persetujuan ketua
`SYSTEM_PROMPT.md`	Luhung	Baca saja — ubah hanya lewat rapat + persetujuan ketua
`config.yaml`	Luhung	Baca saja — ubah hanya lewat rapat + persetujuan ketua
`README.md`	Luhung	Baca saja — ubah hanya lewat rapat + persetujuan ketua
`requirements.txt`	Luhung	Baca saja — ubah hanya lewat rapat + persetujuan ketua

File / Folder	Pemilik	Hak Anggota Lain
`.gitignore`	Luhung	Baca saja — usul perubahan via issue ke ketua
`model/__init__.py` , `model/model.py`	Luhung	Baca saja — usul perubahan via PR review
`data/data.py` , `data/tokenizer.py`	Bagas	Baca saja — serahkan bahan/perubahan ke Bagas
`training/train.py` , `training/metrics.py` , `training/utils.py`	Bima	Baca saja — serahkan bahan/perubahan ke Bima
`outputs/figures/`	Bima	Baca saja (gambar hasil di-commit)
`checkpoints/` , `outputs/logs/`	Bima	Tidak boleh ubah (gitignored, artefak lokal)
`docs/log.md` , `docs/laporan.md` , `docs/llm_chats/`	Ajenk	Baca saja — serahkan bahan ke Ajenk
`data/raw/` , `data/processed/`	Bagas	Tidak boleh ubah (gitignored)

File yang HANYA boleh diubah lewat rapat + persetujuan ketua:

`SPEC.md` , `SYSTEM_PROMPT.md` , `config.yaml` , `README.md` , `requirements.txt` .

Aturan tambahan: perubahan interface lintas modul (nama variabel / shape tensor sesuai SPEC.md §4) wajib lewat PR + review ketua, meskipun file-nya milik anggota lain.

5. Isi `.gitignore`

```
gitignore

# ===== Data & artefak besar (JANGAN commit) =====
data/raw/
```

```
data/processed/
checkpoints/
*.pt
*.pth
outputs/logs/

# ===== Python =====
__pycache__/
*.pyc
*.pyo
*.egg-info/
.venv/
venv/

# ===== Notebook & OS =====
.ipynb_checkpoints/
.DS_Store
Thumbs.db

# ===== Editor =====
.vscode/
.idea/

# ===== Environment & secret =====
.env
*.log
```

File yang TETAP di-commit:

- `SPEC.md`, `SYSTEM_PROMPT.md`, `README.md`, `config.yaml`, `requirements.txt`
- Semua file `.py` (`data/*.py`, `model/*.py`, `training/*.py`)
- `outputs/figures/*.png` (grafik hasil training)
- `docs/*.md` (`log.md`, `laporan.md`)
- `docs/llm_chats/` (transkrip chat LLM)
- Semua `.gitkeep`
- `notebooks/*.ipynb` (jika ada eksperimen per anggota)

6. Aturan Git Tim Paralel

6.1 Konvensi Nama Branch

- `main` — branch produksi, protected, hanya di-merge via Pull Request.
- `feat/luhung-model` — Luhung
- `feat/bagas-data` — Bagas
- `feat/bima-training` — Bima
- `feat/ajenk-docs` — Ajenk
- `fix/<nama>-<deskripsi>` — perbaikan bug
- `docs/<nama>-<deskripsi>` — perubahan dokumentasi

6.2 Konvensi Commit Message

Format: `<tipe>: <deskripsi singkat>`
Tipe: `feat` | `fix` | `docs` | `chore` | `refactor` | `test`

Contoh:

- `feat: tambah MultiHeadAttention dua arah di model.py` — Luhung
- `feat: implementasi masking 80/10/10 di data.py` — Bagas
- `fix: perbaiki gradient clip agar tidak NaN saat warmup` — Bima
- `docs: catat sesi LLM penyusunan struktur folder repo` — Ajenk

6.3 File Rawan Konflik + Solusi

File	Risiko	Solusi
`config.yaml`	Semua butuh ubah hyperparameter	Hanya Luhung edit; usul via issue/rapat
`README.md`	Sering update status	Hanya Luhung edit; PR terpisah dari PR kode
`requirements.txt`	Anggota tambah library	Hanya Luhung edit; usul via issue + persetujuan

File	Risiko	Solusi
`notebooks/*.ipynb`	JSON notebook mudah konflik	Satu file per anggota; jangan edit notebook orang lain

6.4 Alur Kerja Git Tiap Anggota

```bash
# 1. Clone sekali di awal
git clone https://github.com/<org>/bert-from-scratch.git
cd bert-from-scratch

# 2. Sinkronkan main sebelum mulai kerja
git checkout main
git pull origin main

# 3. Buat branch kerja
git checkout -b feat/<nama>-<modul>
# contoh: git checkout -b feat/bagas-data

# 4. Kerja, commit kecil & sering
git add data/data.py
git commit -m "feat: implementasi masking 80/10/10 di data.py"

# 5. Rebase ke main terbaru sebelum push
git fetch origin
git rebase origin/main
# selesaikan konflik jika ada, lalu:
git add <file-yang-konflik>
git rebase --continue

# 6. Push branch
git push origin feat/<nama>-<modul>

# 7. Buka Pull Request ke main via GitHub UI
#    - Reviewer wajib: Luhung (ketua)
#    - Sertakan deskripsi: apa yang berubah, file apa, alasan

# 8. Setelah PR di-merge, sinkronkan lokal
git checkout main
```

```
git pull origin main
git branch -d feat/<nama>-<modul>
```

6.5 Aturan Wajib

- Jangan pernah commit langsung ke `main`.
- Selalu lewat Pull Request; minimal 1 reviewer (Luhung).
- Selalu `git pull --rebase` sebelum `git push`.
- Jangan force-push ke branch bersama.
- Jangan commit file yang ada di `.gitignore`.

7. Justifikasi Struktur

Pemisahan folder per pemilik meminimalkan konflik Git. Setiap anggota punya "wilayah" eksklusif: Bagas di `data/`, Bima di `training/` + `outputs/`, Ajenk di `docs/`, Luhung di `model/` + file root. Konflik merge hampir nol karena tidak ada dua anggota yang rutin menulis file yang sama — kecuali file yang sengaja dipusatkan (`config.yaml`, `README.md`, `requirements.txt`) dan dilindungi aturan "hanya lewat rapat". Ini langsung mendukung penilaian **Kerjasama Tim**.

Memudahkan integrasi modul. Struktur `data/data.py` → `model/model.py` → `training/train.py` mencerminkan alur data di SPEC.md §4. Dengan `__init__.py` di `model/` dan `training/`, import antar modul bersih (`from model import BERT`, `from training.train import train_one_epoch`) tanpa path relatif berlapis. Interface shape `(B, L)` untuk input dan `(B, L, V)` untuk logits menjadi kontrak yang bisa dicek di PR. Ini mendukung penilaian **Manajemen Integrasi**.

Parameter terpusat di `config.yaml`. Semua hyperparameter (`hidden_size=256`, `num_layers=6`, `num_heads=8`, `batch_size=32`, `lr=5e-4`) dan path (`data/raw/`, `checkpoints/`) ada di satu file. Tidak ada hardcode di `data.py`, `model.py`, atau `train.py`. Konsekuensi: (a) mengubah satu angka cukup di satu tempat — sesuai SPEC.md §7 kategori (a); (b) ketidakcocokan parameter antar modul langsung terlihat saat review; (c) LLM punya satu sumber kebenaran saat membantu debugging.

Melindungi repo dari file besar via `.gitignore`. WikiText-2 mentah (~10MB) dan checkpoint (`.pt`, ~80MB per file) akan membuat repo bengkak dan berisiko ditolak GitHub. `.gitignore` memblokir `data/raw/`, `data/processed/`, `checkpoints/`, `*.pt`, `*.pth`, dan `outputs/logs/`. Yang tetap di-commit justru **artefak ringan yang bernilai bukti**: `outputs/figures/*.png` (grafik loss/accuracy untuk laporan) dan `docs/*.md` (log & laporan). Repo tetap ringan, tapi riwayat kerja tetap lengkap — penting untuk penilaian **Kerjasama Tim** (bukti kontribusi) dan **Manajemen Integrasi** (artefak dapat direproduksi).

Kesimpulan: struktur ini adalah keputusan final yang konsisten dengan SPEC.md v1.0 dan aturan perubahan §7. Setiap penyimpangan (menambah folder baru, mengubah pemilik file, mengubah `.gitignore`) wajib lewat rapat kelompok + persetujuan ketua, dan dicatat di `docs/log.md` oleh Ajenk.

Akhir dokumen.

OPSI 2 — Skrip Python untuk Menghasilkan `.docx` Otomatis

Jika kamu lebih suka file `.docx` langsung jadi, jalankan skrip ini di komputer (butuh Python + library `python-docx`). Hasilnya: `Panduan_Struktur_Folder_BERT.docx`.

```
python

# simpan sebagai: generate_docx.py
# jalankan: pip install python-docx
#           python generate_docx.py

from docx import Document
from docx.shared import Pt, RGBColor, Inches
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.enum.style import WD_STYLE_TYPE

doc = Document()

# --- Style dasar ---
style = doc.styles['Normal']
style.font.name = 'Calibri'
style.font.size = Pt(11)
```

```python
# --- Judul ---
title = doc.add_heading('PANDUAN STRUKTUR FOLDER REPO', level=0)
subtitle = doc.add_heading('Proyek: BERT-from-Scratch (Generative AI)', level=1)
doc.add_paragraph('Kelompok: [Nama Kelompok] — Versi Final').italic = True
doc.add_paragraph('Disusun oleh: Luhung (Ketua / Prompt Engineer & LLM Navigator)')
doc.add_paragraph('Tanggal: [DD/MM/YYYY]')
doc.add_paragraph('Status: FINAL — disepakati lewat rapat kelompok')
doc.add_paragraph('Referensi: SPEC.md v1.0')

# --- 1. Tujuan ---
doc.add_heading('1. Tujuan Dokumen', level=1)
doc.add_paragraph(
    'Dokumen ini menetapkan struktur folder final repo GitHub proyek '
    'BERT-from-Scratch sebelum anggota mulai commit. Tujuannya:'
)
for item in [
    'Mencegah bentrok file antar anggota yang bekerja paralel.',
    'Memperjelas batas kepemilikan tiap file.',
    'Menjaga integrasi modul (data.py → model.py → train.py) berjalan mulus.',
    'Melindungi repo dari file besar (dataset, checkpoint) lewat .gitignore.',
]:
    doc.add_paragraph(item, style='List Bullet')
doc.add_paragraph(
    'Struktur ini FINAL dan hanya boleh diubah lewat rapat kelompok + persetujuan ketua.'
).bold = True

# --- 2. Pembagian Peran ---
doc.add_heading('2. Pembagian Peran', level=1)
table = doc.add_table(rows=1, cols=3)
table.style = 'Light Grid Accent 1'
hdr = table.rows[0].cells
hdr[0].text = 'Anggota'; hdr[1].text = 'Peran'; hdr[2].text = 'File Utama'
rows = [
    ('Luhung', 'Ketua / Prompt Engineer', 'SPEC.md, SYSTEM_PROMPT.md, config.yaml, model/
model.py, file root'),
    ('Bagas', 'Data Pipeline / Preprocessing', 'data/data.py, data/tokenizer.py, data/ra
w/, data/processed/'),
    ('Bima', 'Training Loop / Tuning', 'training/train.py, training/metrics.py, training/
utils.py, outputs/'),
    ('Ajenk', 'Documentarian', 'docs/log.md, docs/laporan.md, docs/llm_chats/'),
]
for r in rows:
    cells = table.add_row().cells
```

```
    for i, val in enumerate(r):
        cells[i].text = val

# --- 3. Pohon Direktori ---
doc.add_heading('3. Pohon Direktori Final', level=1)
tree = """bert-from-scratch/
├── README.md                          # [LUHUNG] ringkasan proyek, cara run, status
├── .gitignore                         # [LUHUNG] aturan ignore
├── requirements.txt                   # [LUHUNG] dependensi (torch, datasets, dll)
├── SPEC.md                            # [LUHUNG] spesifikasi teknis final + justifikasi
├── SYSTEM_PROMPT.md                   # [LUHUNG] template prompt standar kelompok
├── config.yaml                        # [LUHUNG] semua hyperparameter & path
│
├── data/
│   ├── raw/                           # [BAGAS] dataset mentah (gitignored)
│   │   └── .gitkeep
│   ├── processed/                     # [BAGAS] hasil tokenisasi & masking (gitignored)
│   │   └── .gitkeep
│   ├── data.py                        # [BAGAS] Dataset, DataLoader, masking pipeline
│   └── tokenizer.py                   # [BAGAS] build_tokenizer, encode/decode, vocab
│
├── model/
│   ├── __init__.py                    # [LUHUNG] ekspor class publik
│   └── model.py                       # [LUHUNG] MultiHeadAttention, EncoderLayer,
│                                      #          BERTEncoder, MLMHead, BERT
│
├── training/
│   ├── __init__.py                    # [BIMA] ekspor fungsi publik training
│   ├── train.py                       # [BIMA] train_one_epoch, evaluate, main loop
│   ├── metrics.py                     # [BIMA] MLM accuracy, perplexity, loss tracker
│   └── utils.py                       # [BIMA] seed, checkpoint save/load, logging help
er
│
├── checkpoints/                       # [BIMA] output .pt/.pth (gitignored)
│   └── .gitkeep
│
├── outputs/
│   ├── figures/                       # [BIMA] grafik loss/accuracy (.png, DI-COMMIT)
│   │   └── .gitkeep
│   └── logs/                          # [BIMA] training logs mentah (gitignored)
│       └── .gitkeep
│
└── docs/
    ├── log.md                         # [AJENK] log kronologis semua sesi LLM
```

```
    ├── laporan.md                    # [AJENK] laporan akhir proyek
    └── llm_chats/                    # [AJENK] arsip transkrip chat LLM per sesi
        └── .gitkeep"""
p = doc.add_paragraph()
run = p.add_run(tree)
run.font.name = 'Consolas'
run.font.size = Pt(8.5)

doc.add_paragraph(
    'Catatan .gitkeep: file kosong untuk menandai folder yang harus tetap ada di Git '
    'meskipun isinya di-ignore.'
).italic = True

# --- 4. Tabel Kepemilikan ---
doc.add_heading('4. Tabel Kepemilikan File', level=1)
table = doc.add_table(rows=1, cols=3)
table.style = 'Light Grid Accent 1'
hdr = table.rows[0].cells
hdr[0].text = 'File / Folder'; hdr[1].text = 'Pemilik'; hdr[2].text = 'Hak Anggota Lain'
owner_rows = [
    ('SPEC.md', 'Luhung', 'Baca saja — ubah hanya lewat rapat + persetujuan ketua'),
    ('SYSTEM_PROMPT.md', 'Luhung', 'Baca saja — ubah hanya lewat rapat + persetujuan ketu
a'),
    ('config.yaml', 'Luhung', 'Baca saja — ubah hanya lewat rapat + persetujuan ketua'),
    ('README.md', 'Luhung', 'Baca saja — ubah hanya lewat rapat + persetujuan ketua'),
    ('requirements.txt', 'Luhung', 'Baca saja — ubah hanya lewat rapat + persetujuan ketu
a'),
    ('.gitignore', 'Luhung', 'Baca saja — usul perubahan via issue ke ketua'),
    ('model/__init__.py, model/model.py', 'Luhung', 'Baca saja — usul perubahan via PR re
view'),
    ('data/data.py, data/tokenizer.py', 'Bagas', 'Baca saja — serahkan bahan ke Bagas'),
    ('training/train.py, metrics.py, utils.py', 'Bima', 'Baca saja — serahkan bahan ke Bi
ma'),
    ('outputs/figures/', 'Bima', 'Baca saja'),
    ('checkpoints/, outputs/logs/', 'Bima', 'Tidak boleh ubah (gitignored)'),
    ('docs/log.md, laporan.md, llm_chats/', 'Ajenk', 'Baca saja — serahkan bahan ke Ajen
k'),
    ('data/raw/, data/processed/', 'Bagas', 'Tidak boleh ubah (gitignored)'),
]
for r in owner_rows:
    cells = table.add_row().cells
    for i, val in enumerate(r):
        cells[i].text = val
```

```
doc.add_paragraph()
doc.add_paragraph(
    'File yang HANYA boleh diubah lewat rapat + persetujuan ketua: '
    'SPEC.md, SYSTEM_PROMPT.md, config.yaml, README.md, requirements.txt.'
).bold = True

# --- 5. .gitignore ---
doc.add_heading('5. Isi .gitignore', level=1)
gitignore = """# ===== Data & artefak besar (JANGAN commit) =====
data/raw/
data/processed/
checkpoints/
*.pt
*.pth
outputs/logs/

# ===== Python =====
__pycache__/
*.pyc
*.pyo
*.egg-info/
.venv/
venv/

# ===== Notebook & OS =====
.ipynb_checkpoints/
.DS_Store
Thumbs.db

# ===== Editor =====
.vscode/
.idea/

# ===== Environment & secret =====
.env
*.log"""
p = doc.add_paragraph()
run = p.add_run(gitignore)
run.font.name = 'Consolas'
run.font.size = Pt(9)

doc.add_paragraph('File yang TETAP di-commit:').bold = True
for item in [
    'SPEC.md, SYSTEM_PROMPT.md, README.md, config.yaml, requirements.txt',
```

```
    'Semua file .py (data/*.py, model/*.py, training/*.py)',
    'outputs/figures/*.png (grafik hasil training)',
    'docs/*.md (log.md, laporan.md)',
    'docs/llm_chats/ (transkrip chat LLM)',
    'Semua .gitkeep',
    'notebooks/*.ipynb (jika ada eksperimen per anggota)',
]:
    doc.add_paragraph(item, style='List Bullet')

# --- 6. Aturan Git ---
doc.add_heading('6. Aturan Git Tim Paralel', level=1)

doc.add_heading('6.1 Konvensi Nama Branch', level=2)
for b in [
    'main — branch produksi, protected, hanya di-merge via Pull Request.',
    'feat/luhung-model — Luhung',
    'feat/bagas-data — Bagas',
    'feat/bima-training — Bima',
    'feat/ajenk-docs — Ajenk',
    'fix/<nama>-<deskripsi> — perbaikan bug',
    'docs/<nama>-<deskripsi> — perubahan dokumentasi',
]:
    doc.add_paragraph(b, style='List Bullet')

doc.add_heading('6.2 Konvensi Commit Message', level=2)
doc.add_paragraph('Format: <tipe>: <deskripsi singkat>')
doc.add_paragraph('Tipe: feat | fix | docs | chore | refactor | test')
doc.add_paragraph('Contoh:')
for c in [
    'feat: tambah MultiHeadAttention dua arah di model.py — Luhung',
    'feat: implementasi masking 80/10/10 di data.py — Bagas',
    'fix: perbaiki gradient clip agar tidak NaN saat warmup — Bima',
    'docs: catat sesi LLM penyusunan struktur folder repo — Ajenk',
]:
    doc.add_paragraph(c, style='List Bullet')

doc.add_heading('6.3 File Rawan Konflik + Solusi', level=2)
table = doc.add_table(rows=1, cols=3)
table.style = 'Light Grid Accent 1'
hdr = table.rows[0].cells
hdr[0].text = 'File'; hdr[1].text = 'Risiko'; hdr[2].text = 'Solusi'
conflict_rows = [
    ('config.yaml', 'Semua butuh ubah hyperparameter', 'Hanya Luhung edit; usul via issu
e/rapat'),
```

```
    ('README.md', 'Sering update status', 'Hanya Luhung edit; PR terpisah dari PR kode'),
    ('requirements.txt', 'Anggota tambah library', 'Hanya Luhung edit; usul via issue + p
ersetujuan'),
    ('notebooks/*.ipynb', 'JSON notebook mudah konflik', 'Satu file per anggota; jangan e
dit notebook orang lain'),
]
for r in conflict_rows:
    cells = table.add_row().cells
    for i, val in enumerate(r):
        cells[i].text = val

doc.add_heading('6.4 Alur Kerja Git Tiap Anggota', level=2)
workflow = """# 1. Clone sekali di awal
git clone https://github.com/<org>/bert-from-scratch.git
cd bert-from-scratch

# 2. Sinkronkan main sebelum mulai kerja
git checkout main
git pull origin main

# 3. Buat branch kerja
git checkout -b feat/<nama>-<modul>

# 4. Kerja, commit kecil & sering
git add data/data.py
git commit -m "feat: implementasi masking 80/10/10 di data.py"

# 5. Rebase ke main terbaru sebelum push
git fetch origin
git rebase origin/main

# 6. Push branch
git push origin feat/<nama>-<modul>

# 7. Buka Pull Request ke main via GitHub UI
#    Reviewer wajib: Luhung (ketua)

# 8. Setelah PR di-merge, sinkronkan lokal
git checkout main
git pull origin main
git branch -d feat/<nama>-<modul>"""
p = doc.add_paragraph()
run = p.add_run(workflow)
run.font.name = 'Consolas'
```

```
run.font.size = Pt(9)

doc.add_heading('6.5 Aturan Wajib', level=2)
for a in [
    'Jangan pernah commit langsung ke main.',
    'Selalu lewat Pull Request; minimal 1 reviewer (Luhung).',
    'Selalu git pull --rebase sebelum git push.',
    'Jangan force-push ke branch bersama.',
    'Jangan commit file yang ada di .gitignore.',
]:
    doc.add_paragraph(a, style='List Bullet')

# --- 7. Justifikasi ---
doc.add_heading('7. Justifikasi Struktur', level=1)
justifikasi = [
    ('Pemisahan folder per pemilik meminimalkan konflik Git. ',
     'Setiap anggota punya wilayah eksklusif: Bagas di data/, Bima di training/ + output
s/, '
     'Ajenk di docs/, Luhung di model/ + file root. Konflik merge hampir nol karena tidak
ada '
     'dua anggota yang rutin menulis file yang sama — kecuali file yang sengaja dipusatka
n '
     '(config.yaml, README.md, requirements.txt) dan dilindungi aturan "hanya lewat rapa
t". '
     'Ini mendukung penilaian Kerjasama Tim.'),
    ('Memudahkan integrasi modul. ',
     'Struktur data/data.py → model/model.py → training/train.py mencerminkan alur data d
i '
     'SPEC.md §4. Dengan __init__.py di model/ dan training/, import antar modul bersih t
anpa '
     'path relatif berlapis. Interface shape (B, L) untuk input dan (B, L, V) untuk logit
s '
     'menjadi kontrak yang bisa dicek di PR. Ini mendukung penilaian Manajemen Integras
i.'),
    ('Parameter terpusat di config.yaml. ',
     'Semua hyperparameter dan path ada di satu file. Tidak ada hardcode di data.py, mode
l.py, '
     'atau train.py. Mengubah satu angka cukup di satu tempat — sesuai SPEC.md §7 kategor
i (a); '
     'ketidakcocokan parameter antar modul langsung terlihat saat review; LLM punya satu
sumber '
     'kebenaran saat membantu debugging.'),
    ('Melindungi repo dari file besar via .gitignore. ',
     'WikiText-2 mentah (~10MB) dan checkpoint (.pt, ~80MB per file) akan membuat repo be
```

```
ngkak '
     'dan berisiko ditolak GitHub. .gitignore memblokir data/raw/, data/processed/, check
points/, '
     '*.pt, *.pth, dan outputs/logs/. Yang tetap di-commit justru artefak ringan yang ber
nilai '
     'bukti: outputs/figures/*.png dan docs/*.md. Repo tetap ringan, riwayat kerja tetap
lengkap.'),
]
for bold_part, rest in justifikasi:
    p = doc.add_paragraph()
    run = p.add_run(bold_part)
    run.bold = True
    p.add_run(rest)

doc.add_paragraph()
doc.add_paragraph(
    'Kesimpulan: struktur ini adalah keputusan final yang konsisten dengan SPEC.md v1.0 d
an '
    'aturan perubahan §7. Setiap penyimpangan wajib lewat rapat kelompok + persetujuan ke
tua, '
    'dan dicatat di docs/log.md oleh Ajenk.'
).bold = True

# --- Simpan ---
output_path = 'Panduan_Struktur_Folder_BERT.docx'
doc.save(output_path)
print(f'✅ File berhasil dibuat: {output_path}')
```

Cara jalankan:

```
bash

pip install python-docx
python generate_docx.py
```

File `Panduan_Struktur_Folder_BERT.docx` akan muncul di folder yang sama dengan skrip. Bisa langsung kamu download/upload ke Google Drive atau grup.

Rekomendasi saya: pakai **Opsi 2** karena hasilnya rapi (heading, tabel, code block ber-font Consolas) dan bisa diregenerasi kapan saja jika ada revisi. Kalau malas install

Python, **Opsi 1** sudah cukup — tinggal copy-paste ke Word.

> Project ini rencananya akan memakai google colab untuk menjalankan sistem dan membangun sistem, dari sini apakkah
>
> 6.4 Alur Kerja Git Tiap Anggota
> bash
> # 1. Clone sekali di awal
> git clone https://github.com/<org>/bert-fro
> cd bert-from-scratch

Analisis: Apakah Alur Git §6.4 Perlu Berubah untuk Google Colab?

Jawaban singkat: Tidak, alur Git-nya tetap sama secara konsep, tetapi **ada beberapa penyesuaian penting** yang harus ditambahkan karena Colab bukan environment lokal biasa (ephemeral, tanpa persistensi, tanpa Git credential default).

Berikut penjelasan lengkap + **versi final §6.4 yang sudah disesuaikan untuk Colab**, siap dimasukkan ke README.

1. Mengapa Alur Inti Tidak Berubah

Perintah `clone → checkout -b → commit → rebase → push → PR → merge` adalah **alur Git standar** yang berjalan identik di Colab, laptop, atau server. Colab punya Git terinstal (`git --version` sudah tersedia). Jadi secara konsep **tidak ada yang salah** dengan §6.4.

2. Apa yang HARUS Disesuaikan untuk Colab

Aspek	Di Laptop Lokal	Di Google Colab	Konsekuensi
Persistensi file	Permanen	**Hilang saat runtime disconnect** (~12 jam idle / 90 menit idle aktif)	Wajib `push` sebelum tutup; clone ulang tiap sesi baru
Git credential	Tersimpan di OS keychain	**Hilang tiap runtime reset**	Wajib pakai Personal Access Token (PAT) atau `gh auth login` tia sesi
Working directory	Pilihan bebas	Default `/content/`	Clone ke `/content/bert-from-scratch`
Google Drive	—	Bisa di-mount untuk persistensi	Direkomendasikan mount Drive agar checkpoint & data tida hilang
Commit author	Sudah di-set `git config` global	**Kosong by default**	Wajib `git config user.name` `user.email` tiap sesi
Rebase interaktif	Aman	Bisa bentrok dengan editor Colab	Hindari `git rebase -i` gunakan `git pull --rebase` nor interaktif
Dataset & checkpoint	Lokal	Hilang saat runtime mati	Simpan di Drive (`/content/drive/MyDrive/...`) atau re-download tia sesi

Kesimpulan: §6.4 perlu **ditambah langkah setup Colab** (autentikasi + config), dan **catatan tentang persistensi** (push sebelum tutup, mount Drive untuk data/checkpoint).

3. Versi Final §6.4 — Siap Masuk README

Ganti §6.4 lama dengan versi di bawah ini. Ini adalah **versi Colab-aware** yang tetap kompatibel dengan laptop lokal (tinggal skip langkah 0).

6.4 Alur Kerja Git Tiap Anggota (Google Colab)

> **Penting:** Google Colab bersifat **ephemeral** — file hilang saat runtime disconnect. Aturannya: **selalu** `push` **sebelum menutup tab Colab**, dan simpan dataset/checkpoint di Google Drive (bukan di `/content/`).

Langkah 0 — Setup awal tiap sesi baru Colab (wajib, sekali per runtime)

```
bash

# 0a. Set identitas Git (hilang tiap runtime reset — wajib diulang)
git config --global user.name "<Nama Kamu>"
git config --global user.email "<email-github-kamu>"

# 0b. Simpan credential agar tidak ditanya berulang (pakai Personal Access Token)
#     Buat PAT di: https://github.com/settings/tokens (scope: repo)
git config --global credential.helper store
# Saat push pertama, masukkan username GitHub + PAT sebagai password

# 0c. (Direkomendasikan) Mount Google Drive untuk checkpoint & data
from google.colab import drive
drive.mount('/content/drive')
```

Langkah 1 — Clone repo (sekali per runtime)

```
bash

# Clone ke /content/ (hilang saat runtime mati, tapi aman karena sudah di-push)
%cd /content
!git clone https://github.com/<org>/bert-from-scratch.git
%cd bert-from-scratch
```

Langkah 2 — Sinkronkan `main` sebelum mulai kerja

bash

```
!git checkout main
!git pull origin main
```

Langkah 3 — Buat branch kerja sesuai peran

bash

```
!git checkout -b feat/<nama>-<modul>
# contoh: !git checkout -b feat/bagas-data
```

Langkah 4 — Kerja & commit kecil-kecil

bash

```
# Edit file (di Colab: double-click file di panel kiri, atau pakai %%writefile)
!git add data/data.py
!git commit -m "feat: implementasi masking 80/10/10 di data.py"
```

Langkah 5 — Rebase ke `main` terbaru sebelum push

bash

```
!git fetch origin
!git rebase origin/main
# Jika ada konflik:
#   1. Edit file yang konflik (panel kiri Colab)
#   2. !git add <file-yang-konflik>
#   3. !git rebase --continue
# Hindari `git rebase -i` di Colab (editor interaktif bermasalah).
```

Langkah 6 — WAJIB: push sebelum tutup Colab

bash

```
!git push origin feat/<nama>-<modul>
# Saat diminta password: masukkan PAT (bukan password GitHub)
```

> ⚠️ **Jika lupa push dan runtime mati → kerjaan hilang permanen.** Tidak ada undo di Colab.

Langkah 7 — Buka Pull Request via GitHub UI

- Buka `https://github.com/<org>/bert-from-scratch` di browser.
- Klik **"Compare & pull request"** untuk branch yang baru di-push.
- **Reviewer wajib: Luhung (ketua).**
- Sertakan deskripsi: apa yang berubah, file apa, alasan.

Langkah 8 — Setelah PR di-merge, sinkronkan

```bash
!git checkout main
!git pull origin main
!git branch -d feat/<nama>-<modul>
```

Langkah 9 — Sebelum menutup Colab (checklist)

□ Semua perubahan sudah `git commit`

□ Branch sudah `git push origin feat/<nama>-<modul>`

□ Checkpoint penting sudah disalin ke `/content/drive/MyDrive/bert-checkpoints/`

□ Dataset processed (jika ada) sudah di Drive atau bisa diregenerasi dari `data.py`

□ Catatan eksperimen sudah ditulis di `docs/log.md` (via Ajenk) atau notebook pribadi

Catatan Tambahan untuk Colab

Situasi	Solusi
Runtime disconnect di tengah training	Simpan checkpoint tiap N step ke Google Drive (`/content/drive/MyDrive/...`), bukan `/content/checkpoints/`
Lupa PAT	Buat ulang di https://github.com/settings/tokens (scope: `repo`)

Situasi	Solusi
`git push` **minta password terus**	Pastikan `git config --global credential.helper store` sudah dijalankan **sebelum** push pertama
Ingin edit file rapi	Pakai panel file Colab (kiri) atau `%%writefile path/file.py` di cell
Butuh lihat diff	`!git diff` atau `!git diff --staged`
Kolaborasi real-time	Colab tidak mendukung multi-user Git; koordinasi via branch + PR tetap wajib

4. Apakah Ini Mengubah README?

Ya — §6.4 di README harus diganti dengan versi di atas. Selain itu, tambahkan **satu bagian baru di README: "Setup Google Colab"** agar anggota baru tidak bingung. Berikut tambahan yang saya sarankan diletakkan **sebelum §6** (atau sebagai §5.5):

Tambahan README — Bagian "Setup Google Colab (Wajib Dibaca)"

````
markdown

## 5.5 Setup Google Colab (Wajib Dibaca Sebelum Mulai)

Proyek ini dijalankan di **Google Colab (T4 GPU)**. Karena Colab bersifat
*ephemeral*, ikuti aturan berikut:

1. **Aktifkan GPU:** Runtime → Change runtime type → T4 GPU.
2. **Set identitas Git tiap sesi baru** (lihat §6.4 Langkah 0).
3. **Mount Google Drive** untuk menyimpan checkpoint & dataset processed:
   ```python
   from google.colab import drive
   drive.mount('/content/drive')
````
````

4. **Simpan checkpoint ke Drive**, bukan `/content/checkpoints/` :

   ```
   python

   CHECKPOINT_DIR = "/content/drive/MyDrive/bert-checkpoints"
   ```

5. **Selalu push sebelum tutup Colab.** File di `/content/` hilang saat runtime mati.
6. **Jangan commit dataset mentah** — `data/raw/` dan `data/processed/` ada di `.gitignore` .
   Dataset diunduh ulang tiap sesi via `data.py` .

Estimasi runtime: 1 epoch MLM di WikiText-2 dengan config SPEC.md ≈ 5–10 menit di T4. 10 epoch ≈ 1–1.5 jam. Colab free tier membatasi ~12 jam per sesi — cukup untuk 1 run penuh, tapi **jangan andalkan persistensi**.

```
text


---

## 5. Ringkasan Jawaban

| Pertanyaan | Jawaban |
|---|---|
| Apakah §6.4 perlu diubah? | **Ya, ditambah** langkah setup Colab (config Git, PAT, moun
t Drive) + peringatan persistensi. Alur inti (clone → branch → commit → rebase → push → P
R) **tetap sama**. |
| Apakah README berubah? | **Ya** — ganti §6.4 dengan versi Colab-aware di atas, dan tamb
ahkan §5.5 "Setup Google Colab". |
| Apakah SPEC.md berubah? | **Tidak.** SPEC.md tidak mengatur Git/Colab. |
| Apakah `.gitignore` berubah? | **Tidak.** Sudah benar (`data/raw/`, `data/processed/`,
`checkpoints/` di-ignore). |
| Apakah struktur folder berubah? | **Tidak.** Hanya cara kerja di Colab yang perlu dicat
at. |

**Rekomendasi:** masukkan versi §6.4 di atas + §5.5 ke README, lalu commit lewat PR `doc
s/luhung-readme-colab` (sesuai aturan §6.3 — README hanya Luhung yang edit). Setelah merg
e, Ajenk mencatat perubahan ini di `docs/log.md`.
```

> buat readme jadi 1 file saja, kemudian ini hanya langkah readme sampai bisa berjalan di colab, untuk aturan commit, merge, pull push tidak perlu

karena itu merupakan pengetahuan umum

README.md — BERT-from-Scratch

markdown

```
# BERT-from-Scratch

Proyek mata kuliah **Generative AI**: membangun model **BERT dari nol** (Masked Language Model) menggunakan PyTorch, dengan metode **Vibe Coding** — seluruh kode, arsitektur, dan debugging dibangun lewat kolaborasi aktif dengan LLM, tanpa copy-paste kode utuh dari internet.

- **Task:** Masked Language Modeling (MLM)
- **Dataset:** WikiText-2 (~2M token)
- **Hardware target:** Google Colab T4 GPU
- **Framework:** PyTorch
- **Spesifikasi teknis lengkap:** lihat [`SPEC.md`](SPEC.md)

---

## 1. Tim & Pembagian Peran

| Anggota | Peran | File Utama |
|---|---|---|
| **Luhung** | Ketua / Prompt Engineer & LLM Navigator | `SPEC.md`, `SYSTEM_PROMPT.md`, `config.yaml`, `model/model.py`, file root |
| **Bagas** | Data Pipeline / Preprocessing | `data/data.py`, `data/tokenizer.py`, `data/raw/`, `data/processed/` |
| **Bima** | Training Loop / Tuning | `training/train.py`, `training/metrics.py`, `training/utils.py`, `outputs/` |
| **Ajenk** | Documentarian | `docs/log.md`, `docs/laporan.md`, `docs/llm_chats/` |

---

## 2. Struktur Folder

```text
bert-from-scratch/
├── README.md                          # [LUHUNG] ringkasan proyek, cara run, status
├── .gitignore                         # [LUHUNG] aturan ignore
├── requirements.txt                   # [LUHUNG] dependensi (torch, datasets, dll)
├── SPEC.md                            # [LUHUNG] spesifikasi teknis final + justifikasi
├── SYSTEM_PROMPT.md                   # [LUHUNG] template prompt standar kelompok
```
```

```
├── config.yaml                        # [LUHUNG] semua hyperparameter & path
│
├── data/
│   ├── raw/                           # [BAGAS] dataset mentah (gitignored)
│   │   └── .gitkeep
│   ├── processed/                     # [BAGAS] hasil tokenisasi & masking (gitignored)
│   │   └── .gitkeep
│   ├── data.py                        # [BAGAS] Dataset, DataLoader, masking pipeline
│   └── tokenizer.py                   # [BAGAS] build_tokenizer, encode/decode, vocab
│
├── model/
│   ├── __init__.py                    # [LUHUNG] ekspor class publik
│   └── model.py                       # [LUHUNG] MultiHeadAttention, EncoderLayer,
│                                      #          BERTEncoder, MLMHead, BERT
│
├── training/
│   ├── __init__.py                    # [BIMA] ekspor fungsi publik training
│   ├── train.py                       # [BIMA] train_one_epoch, evaluate, main loop
│   ├── metrics.py                     # [BIMA] MLM accuracy, perplexity, loss tracker
│   └── utils.py                       # [BIMA] seed, checkpoint save/load, logging help
er
│
├── checkpoints/                       # [BIMA] output .pt/.pth (gitignored)
│   └── .gitkeep
│
├── outputs/
│   ├── figures/                       # [BIMA] grafik loss/accuracy (.png, DI-COMMIT)
│   │   └── .gitkeep
│   └── logs/                          # [BIMA] training logs mentah (gitignored)
│       └── .gitkeep
│
└── docs/
    ├── log.md                         # [AJENK] log kronologis semua sesi LLM
    ├── laporan.md                     # [AJENK] laporan akhir proyek
    └── llm_chats/                     # [AJENK] arsip transkrip chat LLM per sesi
        └── .gitkeep
```

3. Prasyarat

- Akun Google (untuk Colab & Drive)
- Akun GitHub + akses ke repo `<org>/bert-from-scratch`

- **Personal Access Token (PAT) GitHub** dengan scope `repo`
  - Buat di: https://github.com/settings/tokens → *Generate new token (classic)* → centang `repo`
  - Simpan token ini — dipakai sebagai "password" saat `git push` di Colab
- Browser modern (Chrome/Firefox/Edge)

4. Menjalankan Proyek di Google Colab

> **Penting:** Google Colab bersifat *ephemeral* — semua file di `/content/` **hilang** saat runtime disconnect (idle ~90 menit, atau maksimal ~12 jam). Karena itu:
>
> - **Selalu `git push` sebelum menutup tab Colab.**
> - **Simpan checkpoint & dataset processed ke Google Drive**, bukan ke `/content/`.

Ikuti langkah berikut **berurutan** setiap kali memulai sesi Colab baru.

Langkah 1 — Buka Colab & aktifkan GPU

1. Buka https://colab.research.google.com → **New notebook**.
2. Menu **Runtime** → **Change runtime type**.
3. Pilih **Hardware accelerator: T4 GPU** → **Save**.
4. Verifikasi GPU aktif:

```
python
!nvidia-smi
```

Output harus menampilkan `Tesla T4`.

Langkah 2 — Clone repository

Jalankan di cell pertama:

```
python
```

```
%cd /content
!git clone https://github.com/<org>/bert-from-scratch.git
%cd bert-from-scratch
!ls
```

Ganti `<org>` dengan organisasi/username GitHub kelompok.

Langkah 3 — Set identitas Git (wajib tiap sesi baru)

Identitas Git **hilang** setiap runtime reset. Jalankan:

```
python

!git config --global user.name "Nama Kamu"
!git config --global user.email "email@kamu.com"
!git config --global credential.helper store
```

`credential.helper store` membuat Colab mengingat PAT selama runtime aktif (tidak perlu input berulang).

Langkah 4 — Mount Google Drive (untuk checkpoint & dataset)

```
python

from google.colab import drive
drive.mount('/content/drive')
```

Ikuti popup otorisasi. Setelah berhasil, Drive tersedia di `/content/drive/MyDrive/` .

Buat folder persistensi (sekali saja, aman diulang):

```
python

import os
os.makedirs("/content/drive/MyDrive/bert-checkpoints", exist_ok=True)
os.makedirs("/content/drive/MyDrive/bert-data", exist_ok=True)
```

> Semua checkpoint training **wajib** disimpan ke `/content/drive/MyDrive/bert-checkpoints/` , bukan ke

> `/content/bert-from-scratch/checkpoints/` (yang akan hilang).

Langkah 5 — Install dependensi

```
python

%cd /content/bert-from-scratch
!pip install -q -r requirements.txt
```

Verifikasi instalasi:

```
python

import torch, datasets, tokenizers
print("torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

`CUDA available: True` wajib muncul. Jika `False`, ulangi Langkah 1.

Langkah 6 — Unduh dataset

Dataset WikiText-2 diunduh ulang tiap sesi (tidak di-commit, ada di `.gitignore`). Jalankan modul data:

```
python

%cd /content/bert-from-scratch
!python -m data.data --download
```

> **Catatan:** perintah di atas mengasumsikan `data/data.py` punya entry point `--download`. Jika belum, Bagas perlu menambahkannya. Sebagai alternatif sementara, dataset bisa diunduh manual:
>
> ```
> python
>
> from datasets import load_dataset
> ds = load_dataset("wikitext", "wikitext-2-raw-v1")
> print(ds)
> ```

Jika dataset processed sudah ada di Drive dari sesi sebelumnya, salin ke `/content/` agar tidak perlu preprocessing ulang:

```python
!cp -r /content/drive/MyDrive/bert-data/processed /content/bert-from-scratch/data/ 2>/dev/null || echo "Belum ada cache, akan diproses ulang."
```

Langkah 7 — Verifikasi konfigurasi

Buka `config.yaml` dan pastikan path mengarah ke Google Drive untuk checkpoint:

```yaml
# config.yaml (bagian path)
paths:
  data_raw: "data/raw/"
  data_processed: "data/processed/"
  checkpoints: "/content/drive/MyDrive/bert-checkpoints/"
  outputs_figures: "outputs/figures/"
  outputs_logs: "outputs/logs/"
```

> **Penting:** `checkpoints` **harus** mengarah ke Google Drive. `outputs/figures/` tetap di repo (di-commit sebagai bukti hasil).

Langkah 8 — Jalankan training

```python
%cd /content/bert-from-scratch
!python -m training.train --config config.yaml
```

Estimasi waktu di T4: **~5–10 menit per epoch** untuk config SPEC.md (6 layer, hidden 256). 10 epoch ≈ **1–1.5 jam**.

Langkah 9 — Simpan hasil & push sebelum tutup Colab

Setelah training selesai (atau sebelum menutup tab):

```python
# 1. Salin grafik hasil ke folder yang di-commit
!cp /content/drive/MyDrive/bert-checkpoints/*.png outputs/figures/ 2>/dev/null || true

# 2. Commit & push pekerjaan
!git add .
!git commit -m "feat: hasil training run <tanggal>"
!git push origin <nama-branch-kamu>
```

Saat diminta password, masukkan **PAT** (bukan password GitHub).

Checklist sebelum menutup Colab

□ Semua perubahan sudah `git commit`

□ Branch sudah `git push origin <branch>`

□ Checkpoint penting ada di `/content/drive/MyDrive/bert-checkpoints/`

□ Dataset processed (jika ada) ada di `/content/drive/MyDrive/bert-data/`

□ Grafik hasil sudah disalin ke `outputs/figures/` dan di-commit

□ Catatan eksperimen dikirim ke Ajenk untuk `docs/log.md`

5. Menjalankan di Laptop Lokal (Opsional)

Alur sama seperti Colab, tetapi:

- Tidak perlu Langkah 1 (GPU) dan Langkah 4 (mount Drive) — ganti path `checkpoints` di `config.yaml` ke folder lokal.
- Tidak perlu `credential.helper store` jika sudah pakai SSH key atau Git credential manager OS.
- Clone langsung ke folder kerja:

```bash
git clone https://github.com/<org>/bert-from-scratch.git
cd bert-from-scratch
```

```
pip install -r requirements.txt
python -m training.train --config config.yaml
```

6. Troubleshooting Umum di Colab

Masalah	Penyebab	Solusi
`CUDA available: False`	GPU belum aktif	Runtime → Change runtime type → T4 GPU → Save → restart
`git push` minta password terus	`credential.helper` belum di-set	Jalankan `git config --global credential.helper store` **sebelum** push pertama
`Authentication failed` saat push	Pakai password GitHub, bukan PAT	Gunakan **PAT** (scope `repo`) sebagai password
File hilang setelah tab ditutup	Colab ephemeral	Selalu `git push` sebelum tutup; simpan checkpoint ke Drive
`FileNotFoundError: data/raw/...`	Dataset belum diunduh di sesi ini	Jalankan Langkah 6 (unduh dataset)
`ModuleNotFoundError: data`	Working directory salah	`%cd /content/bert-from-scratch` sebelum `python -m ...`
Training OOM (CUDA out of memory)	Batch size terlalu besar	Turunkan `batch_size` di `config.yaml` dari 32 → 16 (perubahan kategori (a) SPEC §7, catat di `docs/log.md`)
Runtime disconnect di tengah training	Idle / limit Colab	Checkpoint otomatis ke Drive; lanjutkan dengan

Masalah	Penyebab	Solusi
		`--resume` (jika didukung `train.py`)

7. Aturan Kolaborasi Singkat

- `main` adalah branch produksi — **jangan commit langsung ke** `main`. Semua perubahan lewat **Pull Request** dan di-review **Luhung**.
- Branch kerja per anggota: `feat/<nama>-<modul>` (contoh: `feat/bagas-data`, `feat/bima-training`).
- File yang **hanya boleh diubah lewat rapat + persetujuan ketua**: `SPEC.md`, `SYSTEM_PROMPT.md`, `config.yaml`, `README.md`, `requirements.txt`.
- Perubahan interface lintas modul (nama variabel / shape tensor sesuai SPEC.md §4) wajib lewat PR + review ketua.
- Semua interaksi dengan LLM dicatat oleh **Ajenk** di `docs/log.md` dan `docs/llm_chats/`.

8. Referensi

- Spesifikasi teknis lengkap: `SPEC.md`
- Template prompt kelompok: `SYSTEM_PROMPT.md`
- Log interaksi LLM: `docs/log.md`
- Laporan akhir: `docs/laporan.md`
- Config hyperparameter: `config.yaml`

Status: 🚧 Dalam pengembangan — SPEC v1.0 final, implementasi modul sedang berjalan.

```text
```

Cara pakai: salin seluruh isi blok ` ```markdown ... ``` ` di atas (tanpa backtick pembungkus) dan simpan sebagai `README.md` di root repo.

Yang berubah dari versi sebelumnya:
- ❌ **Dihapus** bagian aturan commit/merge/pull/push detail (dianggap pengetahuan umum).
- ❌ **Dihapus** tabel file rawan konflik & alur Git bash 8 langkah.
- ✅ **Diganti** dengan **panduan Colab end-to-end** (9 langkah + checklist + troubleshooting).
- ✅ **Ditambah** bagian "Menjalankan di Laptop Lokal" (opsional).
- ✅ **Ditambah** troubleshooting table khusus Colab.
- ✅ **Dipertahankan** struktur folder, pembagian peran, dan aturan kolaborasi singkat (1 paragraf).

> file readme untuk cara kerja Git dari pull sampai bisa berjalan di colab Saja

README.md — Cara Kerja Git & Menjalankan Proyek di Google Colab

markdown

```
# README.md — Cara Kerja Git & Menjalankan Proyek di Google Colab

Dokumen ini memandu alur kerja Git dari **pull** hingga proyek **berjalan di Google Colab**, khusus untuk proyek BERT-from-Scratch.

- **Repo:** `https://github.com/<org>/bert-from-scratch`
- **Framework:** PyTorch
- **Hardware target:** Google Colab T4 GPU
- **Spesifikasi teknis:** lihat [`SPEC.md`](SPEC.md)

---

## 1. Prasyarat

- Akun Google (Colab & Drive)
- Akun GitHub + akses ke repo `<org>/bert-from-scratch`
- **Personal Access Token (PAT)** GitHub dengan scope `repo`
```

  - Buat di: https://github.com/settings/tokens → *Generate new token (classic)* → centang `repo`
  - Simpan token ini — dipakai sebagai "password" saat `git push` di Colab
- Browser modern (Chrome/Firefox/Edge)

> **Penting:** Google Colab bersifat *ephemeral* — file di `/content/` **hilang** saat runtime disconnect (idle ~90 menit, atau maksimal ~12 jam). Karena itu:
> - **Selalu `git push` sebelum menutup tab Colab.**
> - **Simpan checkpoint & dataset processed ke Google Drive**, bukan ke `/content/`.

---

2. Alur Kerja Git (Pull → Branch → Commit → Push → PR)

Alur ini dijalankan **di dalam Colab** setiap sesi kerja. Untuk sesi pertama, mulai dari §3 (setup awal). Untuk sesi berikutnya, cukup ulangi §4 (pull terbaru) → §6 (kerja) → §7 (push).

2.1 Ringkasan Alur

```text
┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ 1. PULL      │──▶│ 2. BRANCH    │──▶│ 3. COMMIT    │──▶│ 4. PUSH      │──▶│ 5. PR        │
│ main terbaru │   │ feat/<...>   │   │ kecil-kecil  │   │ ke origin    │   │ + review     │
└──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘   └──────────────┘
                                                                                   │
                                                                                   ▼
                                                                            ┌──────────────┐
                                                                            │ 6. MERGE     │
                                                                            │ ke main      │
                                                                            └──────────────┘
```

2.2 Perintah Inti (dijalankan di Colab)

```python
# 1. PULL — sinkronkan main terbaru
!git checkout main
!git pull origin main

# 2. BRANCH — buat branch kerja sesuai peran
!git checkout -b feat/<nama>-<modul>
# contoh: !git checkout -b feat/bagas-data
```

```
# 3. COMMIT — commit kecil & sering
!git add data/data.py
!git commit -m "feat: implementasi masking 80/10/10 di data.py"

# 4. PUSH — kirim ke GitHub (masukkan PAT saat diminta password)
!git push origin feat/<nama>-<modul>

# 5. PR — buka Pull Request via GitHub UI
#    https://github.com/<org>/bert-from-scratch
#    Reviewer wajib: Luhung (ketua)

# 6. MERGE — setelah PR di-approve, merge via GitHub UI
#    Lalu sinkronkan lokal:
!git checkout main
!git pull origin main
!git branch -d feat/<nama>-<modul>
```

2.3 Konvensi Branch & Commit

Item	Aturan
Branch produksi	`main` (protected, hanya via PR)
Branch kerja	`feat/<nama>-<modul>` — contoh: `feat/luhung-model`, `feat/bagas-data`, `feat/bima-training`, `feat/ajenk-docs`
Branch perbaikan	`fix/<nama>-<deskripsi>`
Branch dokumentasi	`docs/<nama>-<deskripsi>`
Commit format	`<tipe>: <deskripsi singkat>`
Tipe commit	`feat` | `fix` | `docs` | `chore` | `refactor` | `test`

Contoh commit per anggota:

```
text
```

```
feat: tambah MultiHeadAttention dua arah di model.py          # Luhung
feat: implementasi masking 80/10/10 di data.py                # Bagas
fix: perbaiki gradient clip agar tidak NaN saat warmup        # Bima
docs: catat sesi LLM penyusunan struktur folder repo          # Ajenk
```

2.4 File yang Hanya Boleh Diubah Lewat Rapat + Persetujuan Ketua

`SPEC.md`, `SYSTEM_PROMPT.md`, `config.yaml`, `README.md`, `requirements.txt`.

Perubahan interface lintas modul (nama variabel / shape tensor sesuai SPEC.md §4) wajib lewat PR + review ketua.

3. Setup Awal Colab (Sekali per Runtime)

Jalankan langkah ini **setiap kali memulai sesi Colab baru** (runtime reset = identitas Git & credential hilang).

3.1 Aktifkan GPU

1. Menu **Runtime → Change runtime type**.
2. Pilih **Hardware accelerator: T4 GPU → Save**.
3. Verifikasi:

```
python

!nvidia-smi
```

Output harus menampilkan `Tesla T4`.

3.2 Set Identitas Git

```
python

!git config --global user.name "Nama Kamu"
!git config --global user.email "email@kamu.com"
```

```
!git config --global credential.helper store
```

`credential.helper store` membuat Colab mengingat PAT selama runtime aktif.

3.3 Mount Google Drive (untuk checkpoint & dataset)

```
python

from google.colab import drive
drive.mount('/content/drive')
```

Buat folder persistensi (sekali saja, aman diulang):

```
python

import os
os.makedirs("/content/drive/MyDrive/bert-checkpoints", exist_ok=True)
os.makedirs("/content/drive/MyDrive/bert-data", exist_ok=True)
```

> Semua checkpoint training **wajib** disimpan ke `/content/drive/MyDrive/bert-checkpoints/` , bukan ke `/content/bert-from-scratch/checkpoints/` (yang akan hilang).

4. Clone & Pull Repository

4.1 Clone (sekali per runtime)

```
python

%cd /content
!git clone https://github.com/<org>/bert-from-scratch.git
%cd bert-from-scratch
!ls
```

4.2 Pull `main` terbaru (sebelum mulai kerja)

```python
!git checkout main
!git pull origin main
```

4.3 Buat branch kerja

```python
!git checkout -b feat/<nama>-<modul>
# contoh: !git checkout -b feat/bagas-data
```

5. Install Dependensi

```python
%cd /content/bert-from-scratch
!pip install -q -r requirements.txt
```

Verifikasi:

```python
import torch, datasets, tokenizers
print("torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

`CUDA available: True` wajib muncul. Jika `False`, ulangi §3.1.

6. Siapkan Dataset & Konfigurasi

6.1 Unduh dataset (WikiText-2)

Dataset diunduh ulang tiap sesi (tidak di-commit, ada di `.gitignore`).

```
python

%cd /content/bert-from-scratch
!python -m data.data --download
```

> **Catatan:** perintah di atas mengasumsikan `data/data.py` punya entry point `--download`. Jika belum, Bagas perlu menambahkannya. Alternatif sementara:

```
python

from datasets import load_dataset
ds = load_dataset("wikitext", "wikitext-2-raw-v1")
print(ds)
```

6.2 Restore cache processed dari Drive (jika ada)

```
python

!cp -r /content/drive/MyDrive/bert-data/processed /content/bert-from-scratch/data/ 2>/de
v/null \
  || echo "Belum ada cache, akan diproses ulang."
```

6.3 Verifikasi `config.yaml`

Pastikan path checkpoint mengarah ke Google Drive:

```
yaml

# config.yaml (bagian path)
paths:
  data_raw: "data/raw/"
  data_processed: "data/processed/"
  checkpoints: "/content/drive/MyDrive/bert-checkpoints/"
  outputs_figures: "outputs/figures/"
  outputs_logs: "outputs/logs/"
```

> **Penting:** `checkpoints` **harus** mengarah ke Google Drive. `outputs/figures/` tetap di repo (di-commit sebagai bukti hasil).

7. Jalankan Training

```
python

%cd /content/bert-from-scratch
!python -m training.train --config config.yaml
```

Estimasi waktu di T4: **~5–10 menit per epoch** untuk config SPEC.md (6 layer, hidden 256). 10 epoch ≈ **1–1.5 jam**.

8. Commit & Push Sebelum Tutup Colab

Setelah training selesai (atau sebelum menutup tab):

```
python

# 1. Salin grafik hasil ke folder yang di-commit
!cp /content/drive/MyDrive/bert-checkpoints/*.png outputs/figures/ 2>/dev/null || true

# 2. Commit & push pekerjaan
!git add .
!git commit -m "feat: hasil training run <tanggal>"
!git push origin feat/<nama>-<modul>
```

Saat diminta password, masukkan **PAT** (bukan password GitHub).

Checklist sebelum menutup Colab

□ Semua perubahan sudah `git commit`

□ Branch sudah `git push origin <branch>`

□ Checkpoint penting ada di `/content/drive/MyDrive/bert-checkpoints/`

□ Dataset processed (jika ada) ada di `/content/drive/MyDrive/bert-data/`

□ Grafik hasil sudah disalin ke `outputs/figures/` dan di-commit

□ Catatan eksperimen dikirim ke Ajenk untuk `docs/log.md`

9. Troubleshooting Umum di Colab

Masalah	Penyebab	Solusi
`CUDA available: False`	GPU belum aktif	Runtime → Change runtime type → T4 GPU → Save → restart
`git push` minta password terus	`credential.helper` belum di-set	Jalankan `git config --global credential.helper store` **sebelum** push pertama
`Authentication failed` saat push	Pakai password GitHub, bukan PAT	Gunakan **PAT** (scope `repo`) sebagai password
File hilang setelah tab ditutup	Colab ephemeral	Selalu `git push` sebelum tutup; simpan checkpoint ke Drive
`FileNotFoundError: data/raw/...`	Dataset belum diunduh di sesi ini	Jalankan §6.1
`ModuleNotFoundError: data`	Working directory salah	`%cd /content/bert-from-scratch` sebelum `python -m ...`
Training OOM (CUDA out of memory)	Batch size terlalu besar	Turunkan `batch_size` di `config.yaml` dari 32 → 16 (perubahan kategori (a) SPEC §7, catat di `docs/log.md`)
Runtime disconnect di tengah training	Idle / limit Colab	Checkpoint otomatis ke Drive; lanjutkan dengan `--resume` (jika didukung `train.py`)

10. Referensi

- Spesifikasi teknis lengkap: `SPEC.md`
- Template prompt kelompok: `SYSTEM_PROMPT.md`
- Log interaksi LLM: `docs/log.md`
- Laporan akhir: `docs/laporan.md`
- Config hyperparameter: `config.yaml`

```
text


---

**Cara pakai:** salin seluruh isi blok ` ```markdown ... ``` ` di atas (tanpa backtick pembungkus) dan simpan sebagai `README.md` di root repo.

**Fokus dokumen ini:**
- ✅ Alur Git lengkap: **pull → branch → commit → push → PR → merge** (§2).
- ✅ Langkah **setup Colab** dari nol sampai training berjalan (§3–§7).
- ✅ **Commit & push sebelum tutup** + checklist (§8).
- ✅ Troubleshooting khusus Colab (§9).
- ❌ Tidak ada pembahasan aturan commit/merge/pull/push yang bersifat pengetahuan umum — hanya konvensi singkat yang relevan untuk proyek ini (§2.3).
```

Continue with DeepSeek